Introduction

This book covers the design patterns for *context-generic programming* (CGP), a new programming paradigm for Rust that allows strongly-typed components to be implemented and composed in a modular, generic, and type-safe way.

What is Context-Generic Programming

A high level overview of CGP is available on the project website. This section contains a summarized version of the overview.

At its core, CGP makes use of Rust's trait system to build generic component *interfaces* that decouple code that *consumes* an interface from code that *implements* an interface. Through this decoupling, code can be written to be generic over any context, and then be wired to be used on a concrete context by writing few lines of code. CGP makes use of Rust's strong type system to help ensure that any such wiring is *type-safe*, catching any unsatisfied dependencies as compile-time errors.

CGP shares some similarities with other modular programming patterns, such as OCaml modules, Scala implicits, mixins, and dependency injection. Compared to these other patterns, CGP has a unique advantage that it enables high modularity while also being type-safe and concise. With Rust as its host language, CGP also allows high-performance and low-level code to be written in a modular way, without requiring complex runtime support.

CGP is designed to solve a wide range of common problems in programming, including error handling, logging, encoding, and modular dispatch. In particular, it allows writing static-typed Rust programs to be almost as expressive as writing dynamic-typed programs, but with the additional type safety guarantees. If you ever feel that Rust's type system is restricting your ability to reuse code, or be forced to duplicate code through copy-pasting or macros, then CGP may be of help to solve your problem.

That said, programming in CGP is as expressive, but not as easy, as dynamic-typed programming. There may be a steep learning curve in learning how to program in a generic way, and this book aims to help make that learning process more approachable. Thoughout this book, we will slowly understand how CGP works, and learn about useful design patterns that can be used in any programming situation.

Work In Progress

This book is currently a work in progress. A majority of the chapter is yet to be written. Please come back later to check out a completed version of this book.

Who This Book Is For

This book takes a deep dive into how CGP works, building and explaining its core constructs from the ground up.

However, it's worth noting that many of the concepts explained here are not immediately essential for simply writing working CGP code, particularly when you are just starting out. This is because CGP provides a high level of abstraction, allowing developers to write highly productive code without needing to understand all the underlying details.



If your primary goal is to start writing CGP code and become productive as quickly as possible, this book might not be the most direct path.

On the other hand, this book is primarily intended for those who wish to gain a better understanding of what happens "behind the macros" after they have gained some experience using CGP.

It will also be a valuable resource for users encountering CGP-related error messages, providing the knowledge needed to understand the root cause and fix issues more effectively.

For beginners looking for a faster start, a separate book is planned for the future, which will provide more beginner-friendly tutorials focused on quickly getting started with CGP. book will be written in the future, to provide beginner-friendly tutorials for getting started with CGP more quickly.

Chapter Outlines

The first section of this book, *Terminology*, will introduce common terms to be used to understand CGP. We will learn about what is a context, and what are consumer and provider traits.

In the next section, *Core Concepts*, we will cover the core concepts and the Rust-specific design patterns that we use to enable context-generic programming.

Following that, *Design Patterns* will introduce general design patterns that are built on top of the foundation of context-generic programming.

The section *Domain-Specific Patterns* will cover use-case-specific design patterns, such as

error handling and logging.

Finally, the secion *Related Concepts* will compare context-generic programming with related concepts, such as the similarity and differences of context-generic programming as compared to object-oriented programming.

Contribution

This book is open sourced under the MIT license on GitHub.

Anyone is welcome to contribute by submitting pull requests for grammatical correction, content improvement, or adding new design patterns.

A GitHub Discussions forum is available for readers to ask questions or have discussions for topics covered in this book.

Context

In CGP, we use the term *context* to refer to a *type* that provide certain functionalities, or dependencies. The most common kind of functionality a context may provide is a *method*.

Following is a simple hello world example of a context providing a method:

```
struct MyContext;

impl MyContext {
    fn hello(&self) {
        println!("Hello World!");
    }
}
```

The example above is mostly self explanatory. We first define a struct called `MyContext`, followed by an `impl` block for `MyContext`. Inside the `impl` block, a `hello` method is provided, which prints out `"Hello World!"` to the terminal when called.

We can then use the `hello` method anywhere that we have a value of type `MyContext`, such as:

```
let my_context = MyContext;

my_context.hello();
```

Contexts vs Classes

The above example may seem trivial for most programmers, especially for those who come from object-oriented programming (OOP) background. In fact, one way we can have a simplified view of a context is that it is similar to OOP concepts such as *classes*, *objects*, and *interfaces*.

Beyond the surface-level similarity, the concept of contexts in CGP is more general than classes and other similar concepts. As a result, it is useful to think of the term *context* as a new concept that we will learn in this book.

As we will learn in later chapters, aside from methods, a context may provide other kinds of functionalities, such as *associated types* and *constants*.

Contexts vs Types

Although a context is usually made of a type, in CGP we do not treat all types as contexts. Instead, we expect CGP contexts to offer some level of *modularity*, which can be achived by using the programming patterns introduced in this book.

Consumer

In CGP, a *consumer* is a piece of code that consumes certain functionalities from a context. There are several ways which a consumer may consume a functionality. At its most basic, if a consumer has access to the *concrete* type of a context, it can access any methods defined by an `impl` block of that context.

```
struct Person { name: String }

impl Person {
    fn name(&self) -> &str {
        &self.name
    }
}

fn greet(person: &Person) {
    println!("Hello, {}!", person.name());
}
```

in the above example, we have a `greet` function that prints a greeting to a person using the method `Person::name`. In other words, we say that the `greet` function is a *consumer* to the `Person::name` method.

Context-Generic Consumers

The `greet` function in our previous example can only work with the `Person` struct. However, if we inspect the implementation of `greet`, we can see that it is possible to generalize `greet` to work with *any* type that has a name.

To generalize `greet`, we first need to define a *trait* that acts as an *interface* for getting a name:

```
trait HasName {
    fn name(&self) -> &str;
}

fn greet<Context>(context: &Context)
where
    Context: HasName
{
    println!("Hello, {}", context.name());
}
```

In the example above, we define a `HasName` trait that provides a `name` method. We then redefine `greet` to work generically with any `Context` type, with the `where` clause

requiring `Context` to implement `HasName`. Inside the function body, we call the `name` method, and print out the greeting of that name.

Notice that in this example, we are able to implement `greet` *before* we have any concrete implementation of `HasName`. Compared to before, `greet` is now *decoupled* from the `Person` type, thus making our code more modular.

In CGP, this new version of `greet` is considered a *context-generic* consumer, as it is able to *generically* consume the `HasName::name` method from any `Context` type that implements `HasName`.

The concept of context-generic consumer is not unique to CGP. In fact, it is already commonly used in most of the Rust code that uses traits. However, we make an effort to study this concept, so that we can further generalize the concept in the later chapters of this book.

Provider

In CGP, a *provider* is a piece of code that *implements* certain functionality for a context. At its most basic, a provider is consist of an `impl` block for a trait.

```
trait HasName {
    fn name(&self) -> &str;
}

struct Person { name: String }

impl HasName for Person {
    fn name(&self) -> &str {
        &self.name
    }
}
```

In the above example, we implement the `HasName` for the `Person` struct. The block `impl HasName for Person` is a *provider* of the `HasName` trait for the `Person` context.

Similar to the concept of a consumer, the use of provider is common in any Rust code that implements a trait. However, compared to cosumers, there are limitations on how providers can be defined in Rust.

For this example, the `impl` block is a *context-specific* provider for the `Person` context. Furthermore, due to the restrictions of Rust's trait system, there can be at most one provider of `HasName` for the `Person` context. Another common restriction is that the provider has to be defined in the same crate as either the trait or the context.

The asymetry between what can be done with a provider, as compared to a consumer, is often a source of complexity in many Rust programs. As we will learn in later chapters, one of the goals of CGP is to break this asymetry, and make it easy to implement *context-generic providers*.

Providers as Consumers

Although we have providers and consumers as distinct concepts, it is common to have code that serve as *both* providers and consumers.

```
trait CanGreet {
    fn greet(&self);
}

impl CanGreet for Person {
    fn greet(&self) {
        println!("Hello, {}!", self.name());
    }
}
```

The example above shows a new `CanGreet` trait, which provides a `greet` method. We then implement `CanGreet` for `Person`, with the `greet` implementation using `self.name()` to print out the name to be greeted.

Here, the block `impl CanGreet for Person` is a provider of `CanGreet` for the `Person` context. At the same time, it is also the *consumer* of `HasName` for the `Person` context. In terms of genericity, the example code is *context-specific* to the `Person` context for both the consumer and provider side.

As we will see in later chapters, a powerful idea introduced by CGP is that a piece of code can have *multiple spectrums* of genericity on the consumer and provider sides.

Blanket Trait Implementations

In the previous chapter, we have an implementation of `CanGreet` for `Person` that makes use of `HasName` to retrieve the person's name to be printed. However, the implementation is *context-specific* to the `Person` context, and cannot be reused for other contexts.

Ideally, we want to be able to define *context-generic* implementations of `Greet` that works with any context type that also implements `HasName`. For this, the *blanket trait implementations* pattern is one basic way which we can use for defining context-generic implementations:

```
trait HasName {
    fn name(&self) -> &str;
}

trait CanGreet {
    fn greet(&self);
}

impl<Context> CanGreet for Context
where
    Context: HasName,
{
    fn greet(&self) {
        println!("Hello, {}!", self.name());
    }
}
```

The above example shows a blanket trait implementation of `CanGreet` for any `Context` type that implements `HasName`. With that, contexts like `Person` do not need to explicitly implement `CanGreet`, if they already implement `HasName`:

```
struct Person { name: String }

impl HasName for Person {
    fn name(&self) -> &str {
        &self.name
    }
}

let person = Person { name: "Alice".to_owned() };
person.greet();
```

As shown above, we are able to call `person.greet()` without having a context-specific implementation of `CanGreet` for `Person`.

Extension Traits

The use of blanket trait implementation is commonly found in many Rust libraries today. For example, `Itertools` provides a blanket implementation for any context that implements `Iterator`. Another example is `StreamExt`, which is implemented for any context that implements `Stream`.

Traits such as `Itertools` and `StreamExt` are sometimes known as *extension traits*. This is because the purpose of the trait is to *extend* the behavior of existing types, without having to own the type or base traits. While the use of extension traits is a common use case for blanket implementations, there are other ways we can make use of blanket implementations.

Overriding Blanket Implementations

Traits containing blanket implementation are usually not meant to be implemented manually by individual contexts. They are usually meant to serve as convenient methods that extends the functionality of another trait. However, Rust's trait system does *not* completely prevent us from overriding the blanket implementation.

Supposed that we have a `VipPerson` context that we want to implement a different way of greeting the VIP person. We could override the implementation as follows:

```
trait HasName {
    fn name(&self) -> &str;
}

trait CanGreet {
    fn greet(&self);
}

impl<Context> CanGreet for Context
where
    Context: HasName,
{
    fn greet(&self) {
        println!("Hello, {}!", self.name());
    }
}

struct VipPerson { name: String, /* other fields */ }

impl CanGreet for VipPerson {
    fn greet(&self) {
        println!("A warm welcome to you, {}!", self.name);
    }
}
```

The example above shows *two* providers of `CanGreet`. The first provider is a context-generic provider that we covered previously, but the second provider is a context-specific provider for the `VipPerson` context.

Conflicting Implementations

In the previous example, we are able to define a custom provider for `VipPerson`, but with an important caveat: that `VipPerson` does *not* implement `HasName`. If we try to define a custom provider for contexts that already implement `HasName`, such as for `Person`, the compilation would fail:

```
trait HasName {
    fn name(&self) -> &str;
}

trait CanGreet {
    fn greet(&self);
}

impl<Context> CanGreet for Context
where
    Context: HasName,
{
    fn greet(&self) {
        println!("Hello, {}!", self.name());
    }
}

struct Person { name: String }

impl HasName for Person {
    fn name(&self) -> &str {
        &self.name
    }
}

impl CanGreet for Person {
    fn greet(&self) {
        println!("Hi, {}!", self.name());
    }
}
```

If we try to compile the example code above, we would get an error with the message:

```
conflicting implementations of trait `CanGreet` for type `Person`
```

The reason for the conflict is because Rust trait system requires all types to have unambigious implementation of any given trait. To see why such requirement is

necessary, consider the following example:

```
fn call_greet_generically<Context>(context: &Context)
where
    Context: HasName,
{
    context.greet()
}
```

The example above shows a generic function `call_greet_generically`, which work with any `Context` that implements `HasName`. Even though it does not require `Context` to implement `CanGreet`, it nevertheless can call `context.greet()`. This is because with the guarantee from Rust's trait system, the compiler can always safely use the blanket implementation of `CanGreet` during compilation.

If Rust were to allow ambiguous override of blanket implementations, such as what we tried with `Person`, it would have resulted in inconsistencies in the compiled code, depending on whether it is known that the generic type is instantiated to `Person`.

Note that in general, it is not always possible to know locally about the concrete type that is instantiated in a generic code. This is because a generic function like `call_greet_generically` can once again be called by other generic code. This is why even though there are unstable Rust features such as *trait specialization*, such feature has to be carefully considered to ensure that no inconsistency can arise.

Limitations of Blanket Implementations

Due to potential conflicting implementations, the use of blanket implementations offer limited customizability, in case if a context wants to have a different implementation. Although a context many define its own context-specific provider to override the blanket provider, it would face other limitations such as not being able to implement other traits that may cause a conflict.

In practice, we consider that blanket implementations allow for *a singular context-generic provider* to be defined. In future chapters, we will look at how to relax the singular constraint, to make it possible to allow *multiple* context-generic or context-specific providers to co-exist.

Impl-side Dependencies

When writing generic code, we often need to specify the trait bounds that we would like to use with a generic type. However, when the trait bounds involve traits that make use of blanket implementations, there are different ways that we can specify the trait bounds.

Supposed that we want to define a generic function that formats a list of items into a comma-separated string. Our generic function could make use the `Itertools::join` to format an iterator. Our first attempt would be to define our generic function as follows:

```
use core::fmt::Display;
use itertools::Itertools;

fn format_iter<I>(mut iter: I) -> String
where
    I: Iterator,
    I::Item: Display,
{
    iter.join(", ")
}

assert_eq!(format_iter(vec![1, 2, 3].into_iter()), "1, 2, 3");
```

The generic function `format_iter` takes a generic type `I` that implements `Iterator`. Additionally, we require `I::Item` to implement `Display`. With both constraints in place, we are able to call `Itertools::join` inside the generic function to join the items using `", "` as separator.

In the above example, we are able to use the method from `Itertools` on `I`, even though we do not specify the constraint `I: Itertools` in our `where` clause. This is made possible because the trait `Itertools` has a blanket implementation on all types that implement `Iterator`, including the case when we do not know the concrete type behind `I`. Additionally, the method `Itertools::join` requires `I::Item` to implement `Display`, so we also include the constraint in our where clause.

When using traits that have blanket implementation, we can also go the other way and require `I` to implement `Itertools` instead of `Iterator`:

```
fn format_iter<I>(mut items: I) -> String
where
    I: Itertools,
    I::Item: Display,
{
    items.join(", ")
}

assert_eq!(format_iter(vec![1, 2, 3].into_iter()), "1, 2, 3");
```

By doing so, we make it explicit of the intention that we only care that `I` implements `Itertools`, and hide the fact that we need `I` to also implement `Iterator` in order to implement `Itertools`.

Constraint Leaks

At this point, we have defined our generic function `format_iter` with two constraints in the `where` clause. When calling `format_iter` from another generic function, the constraint would also be propagated to the caller.

As a demonstration, supposed that we want to define another generic function that uses `format_iter` to format any type that implements `IntoIterator`, we would need to also include the constraints needed by `format_iter` as follows:

```
fn format_items<C>(items: C) -> String
where
    C: IntoIterator,
    C::IntoIter: Itertools,
    <C::IntoIter as Iterator>::Item: Display,
{
    format_iter(items.into_iter())
}

assert_eq!(format_items(&vec![1, 2, 3]), "1, 2, 3");
```

When defining the generic function `format_items` above, we only really care that the generic type `C` implements `IntoIterator`, and then pass `C::IntoIter` to `format_iter`. However, because of the constraints specified by `format_iter`, Rust also forces us to specify the same constraints in `format_items`, even if we don't need the constraints directly.

As we can see, the constraints specified in the `where` clause of `format_iter` is a form of leaky abstraction, as it also forces generic consumers like `format_items` to also know about the internal details of how `format_iter` uses the iterator.

The leaking of `where` constraints also makes it challenging to write highly generic functions at a larger scale. The number of constraints could quickly become unmanageable, if a high level generic function calls many low-level generic functions that each has different constraints.

Furthermore, the repeatedly specified constraints become tightly coupled with the concrete implementation of the low-level functions. For example, if `format_iter` changed from using `Itertools::join` to other ways of formatting the iterator, the constraints would become outdated and need to be changed in `format_items`.

Hiding Constraints with Traits and Blanket Implementations

Using the techniques we learned from blanket implementations, there is a way to hide the `where` constraints by redefining our generic functions as traits with blanket implementations.

We would first rewrite `format_iter` into a trait `CanFormatIter` as follows:

```
pub trait CanFormatIter {
    fn format_iter(self) -> String;
}

impl<I> CanFormatIter for I
where
    I: Itertools,
    I::Item: Display,
{
    fn format_iter(mut self) -> String
    {
        self.join(", ")
    }
}

assert_eq!(vec![1, 2, 3].into_iter().format_iter(), "1, 2, 3");
```

The trait `CanFormatIter` is defined with a single method, `format_iter`, which consumes `self` and return a `String`. The trait comes with a blanket implementation for any type `I`, with the constraints that `I: Itertools` and `I::Item: Display`. Following that, we have the same implementation as before, which calls `Itertools::join` to format the iterator as a comma-separated string. By having a blanket implementation, we signal that `CanFormatIter` is intended to be derived automatically, and that no explicit implementation is required.

It is worth noting that the constraints `I: Itertools` and `I::Item: Display` are only present at the `impl` block, but not at the `trait` definition of `CanFormatIter`. By doing so, we have effectively "hidden" the constraints inside the `impl` block, and prevent it from leaking to its consumers.

We can now refactor `format_items` to use `CanFormatIter` as follows:

```
fn format_items<C>(items: C) -> String
where
    C: IntoIterator,
    C::IntoIter: CanFormatIter,
{
    items.into_iter().format_iter()
}

assert_eq!(format_items(&vec![1, 2, 3]), "1, 2, 3");
```

In the new version of `format_items`, our `where` constraints are now simplified to only require `C::IntoIter` to implement `CanFormatIter`. With that, we are able to make it explicit that `format_items` needs `CanFormatIter` to be implemented, but it doesn't matter *how* it is implemented.

The reason this technique works is similar to how we used `Itertools` in our previous examples. At the call site of the code that calls `format_items`, Rust would see that the generic function requires `C::IntoIter` to implement `CanFormatIter`. But at the same time, Rust also sees that `CanFormatIter` has a blanket implementation. So if the constraints specified at the blanket implementation are satisfied, Rust would automatically provide an implementation of `CanFormatIter` to `format_items`, without the caller needing to know how that is done.

Nested Constraints Hiding

Once we have seen in action how we can hide constraints behind the blanket `impl` blocks of traits, there is no stopping for us to define more traits with blanket implementations to hide even more constraints.

For instance, we could rewrite `format_items` into a `CanFormatItems` trait as follows:

```rust
pub trait CanFormatItems {
    fn format_items(&self) -> String;
}

impl<Context> CanFormatItems for Context
where
    for<'a> &'a Context: IntoIterator,
    for<'a> <&'a Context as IntoIterator>::IntoIter: CanFormatIter,
{
    fn format_items(&self) -> String
    {
        self.into_iter().format_iter()
    }
}

assert_eq!(vec![1, 2, 3].format_items(), "1, 2, 3");
```

We first define a `CanFormatItems` trait, with a method `format_items(&self)`. Here, we make an improvement over the original function to allow a reference `&self`, instead of an owned value `self`. This allows a container such as `Vec` to not be consumed when we try to format its items, which would be unnecessarily inefficient.

Inside the blanket `impl` block for `CanFormatItems`, we define it to work with any `Context` type, given that the generic `Context` type implements some constraints with *higher ranked trait bounds* (HRTB). While HRTB is an advanced subject on its own, the general idea is that we require that any reference `&'a Context` with any lifetime `'a` implements `IntoIterator`. This is so that when `IntoIterator::into_iter` is called, the `Self` type being consumed is the reference type `&'a Context`, which is implicitly copyable, and thus allow the same context to be reused later on at other places.

Additionally, we require that `<&'a Context as IntoIterator>::IntoIter` implements `CanFormatIter`, so that we can call its method on the produced iterator. Thanks to the hiding of constraints by `CanFormatIter`, we can avoid specifying an overly verbose constraint that the iterator item also needs to implement `Display`.

Individually, the constraints hidden by `CanFormatIter` and `CanFormatItems` may not look significant. But when combining together, we can see how isolating the constraints help us better organize our code and make them cleaner. In particular, we can now write generic functions that consume `CanFormatIter` without having to understand all the indirect constraints underneath.

To demonstrate, supposed that we want to compare two list of items and see whether they have the same string representation. We can now define a generic `stringly_equals` function as follows:

```
fn stringly_equals<Context>(left: &Context, right: &Context) -> bool
where
    Context: CanFormatItems,
{
    left.format_items() == right.format_items()
}

assert_eq!(stringly_equals(&vec![1, 2, 3], &vec![1, 2, 4]), false);
```

Our generic function `stringly_equals` can now be defined cleanly to work over any `Context` type that implements `CanFormatItems`. In this case, the function does not even need to be aware that `Context` needs to produce an iterator, with its items implementing `Display`.

Furthermore, instead of defining a generic function, we could instead use the same programming technique and define a `CanStringlyCompareItems` trait that does the same thing:

```
pub trait CanStringlyCompareItems {
    fn stringly_equals(&self, other: &Self) -> bool;
}

impl<Context> CanStringlyCompareItems for Context
where
    Context: CanFormatItems,
{
    fn stringly_equals(&self, other: &Self) -> bool {
        self.format_items() == other.format_items()
    }
}

assert_eq!(vec![1, 2, 3].stringly_equals(&vec![1, 2, 4]), false);
```

For each new trait we layer on top, we can build higher level interfaces that hide away lower level implementation details. When `CanStringlyCompareItems` is used, the consumer is shielded away from knowing anything about the concrete context, other than that two values are be compared by first being formatted into strings.

The example here may seem a bit stupid, but there are some practical use cases of implementing comparing two values as strings. For instance, a serialization library may want to use it inside tests to check whether two different values are serialized into the same string. For such use case, we may want to define another trait to help make such assertion during tests:

```rust
pub trait CanAssertEqualImpliesStringlyEqual {
    fn assert_equal_implies_stringly_equal(&self, other: &Self);
}

impl<Context> CanAssertEqualImpliesStringlyEqual for Context
where
    Context: Eq + CanStringlyCompareItems,
{
    fn assert_equal_implies_stringly_equal(&self, other: &Self) {
        assert_eq!(self == other, self.stringly_equals(other))
    }
}

vec![1, 2, 3].assert_equal_implies_stringly_equal(&vec![1, 2, 3]);
vec![1, 2, 3].assert_equal_implies_stringly_equal(&vec![1, 2, 4]);
```

The trait `CanAssertEqualImpliesStringlyEqual` provides a method that takes two contexts of the same type, and assert that if both context values are equal, then their string representation are also equal. Inside the blanket `impl` block, we require that `Context` implements `CanStringlyCompareItems`, as well as `Eq`.

Thanks to the hiding of constraints, the trait `CanAssertEqualImpliesStringlyEqual` can cleanly separate its direct dependencies, `Eq`, from the rest of the indirect dependencies.

Dependency Injection

The programming technique that we have introduced in this chapter is sometimes known as *dependency injection* in some other languages and programming paradigms. The general idea is that the `impl` blocks are able to specify the dependencies they need in the form of `where` constraints, and the Rust trait system automatically helps us to resolve the dependencies at compile time.

In context-generic programming, we think of constraints in the `impl` blocks not as constraints, but more generally *dependencies* that the concrete implementation needs. Each time a new trait is defined, it serves as an interface for consumers to include them as a dependency, but at the same time separates the declaration from the concrete implementations.

Provider Traits

In the previous chapters on blanket implementations and impl-side dependencies, we learned about the power of using blanket `impl` blocks to simplify and hide the dependencies required by each part of the implementation. However, one major limitation of blanket implementations is that there cannot be multiple potentially overlapping implementations, due to restrictions in Rust's trait system. In CGP, we can overcome this limitation by introducing the concept of *provider traits*.

The main idea behind provider traits is to define Rust traits that are dedicated for providers to define new implementations, and separate it from the *consumer traits* that are more suitable for consumers that use the traits. Consider a simple consumer trait `CanFormatToString`, which allows formatting a context into string:

```
pub trait CanFormatString {
    fn format_string(&self) -> String;
}
```

The trait we defined here is almost identical to the standard library's `ToString` trait. But we will duplicate the trait here to tweak how it is implemented. We first note that the original `ToString` trait has a blanket implementation for any type that implements `Display`:

```
use core::fmt::Display;

impl<Context> CanFormatString for Context
where
    Context: Display,
{
    fn format_string(&self) -> String {
        format!("{}", self)
    }
}
```

Although having this blanket implementation is convenient, it restricts us from being able to format the context in other ways, such as using `Debug`.

```rust
use core::fmt::{Display, Debug};

impl<Context> CanFormatString for Context
where
    Context: Display,
{
    fn format_string(&self) -> String {
        format!("{}", self)
    }
}

// Error: conflicting implementation
impl<Context> CanFormatString for Context
where
    Context: Debug,
{
    fn format_string(&self) -> String {
        format!("{:?}", self)
    }
}
```

To overcome this limitation, we can introduce a *provider trait* that we'd call `StringFormatter`, which we will then use for defining implementations:

```rust
pub trait StringFormatter<Context> {
    fn format_string(context: &Context) -> String;
}
```

Compared to `CanFormatString`, the trait `StringFormatter` replaces the *implicit* context type `Self` with an *explicit* context type `Context`, as defined in its type parameter. Following that, it replaces all occurrances of `&self` with `context: &Context`.

By avoiding the use of `Self` in provider traits, we can bypass the restrictions of Rust trait system, and have multiple implementations defined. Continuing the earlier example, we can define the `Display` and `Debug` implementations of `CanFormatString` as two separate providers of `StringFormatter`:

```rust
use core::fmt::{Display, Debug};

pub struct FormatStringWithDisplay;

pub struct FormatStringWithDebug;

impl<Context> StringFormatter<Context> for FormatStringWithDisplay
where
    Context: Display,
{
    fn format_string(context: &Context) -> String {
        format!("{}", context)
    }
}

impl<Context> StringFormatter<Context> for FormatStringWithDebug
where
    Context: Debug,
{
    fn format_string(context: &Context) -> String {
        format!("{:?}", context)
    }
}
```

With provider traits, we now have two *named* providers `FormatStringWithDisplay` and `FormatStringWithDebug`, which are defined as dummy structs. These structs are not meant to be used inside any code during run time. Rather, they are used as *identifiers* at the *type level* for us to refer to the providers during compile time.

Notice that inside the implementation of `StringFormatter`, the types `FormatStringWithDisplay` and `FormatStringWithDebug` are in the position that is typically used for `Self`, but we don't use `Self` anywhere in the implementation. Instead, the original `Self` type is now referred explicitly as the `Context` type, and we use `&context` instead of `&self` inside the implementation.

From the point of view of Rust's trait system, the rules for overlapping implementation only applies to the `Self` type. But because we have two distinct `Self` types here (`FormatStringWithDisplay` and `FormatStringWithDebug`), the two implementations are not considered overlapping, and we are able to define them without any compilation error.

Using Provider Traits Directly

Although provider traits allow us to define overlapping implementations, the main downside is that consumer code cannot make use of an implementation without explicitly choosing the implementation.

Consider the following `Person` context defined:

```
use core::fmt::{self, Display, Debug};

#[derive(Debug)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

impl Display for Person {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{} {}", self.first_name, self.last_name)
    }
}

let person = Person { first_name: "John".into(), last_name: "Smith".into() };

assert_eq!(
    FormatStringWithDisplay::format_string(&person),
    "John Smith"
);

assert_eq!(
    FormatStringWithDebug::format_string(&person),
    "Person { first_name: \"John\", last_name: \"Smith\" }"
);
```

Our `Person` struct is defined with both `Debug` and `Display` implementations. When using `format_string` on a value `person: Person`, we cannot just call `person.format_string()`. Instead, we have to explicitly pick a provider `CgpProvider`, and call it with `Provider::format_string(&person)`. On the other hand, thanks to the explicit syntax, we can use both `FormatStringWithDisplay` and `FormatStringWithDebug` on `Person` without any issue.

Nevertheless, having to explicitly pick a provider can be problematic, especially if there are multiple providers to choose from. In the next chapter, we will look at how we can link a provider trait with a consumer trait, so that we can use back the simple `person.format_string()` syntax without needing to know which provider to choose from.

Beyond String Formatting

In this chapter, we make use of a very simplified example of formatting strings to demonstrate the use case of provider traits. Our example may seem a bit redundant, as it does not simplify the code much as compared to directly using `format!()` to format the string with either `Debug` or `Display`. However, the provider trait allows us to also define

providers that format a context in other ways, such as by serializing it as JSON:

```
use anyhow::Error;
use serde::Serialize;
use serde_json::to_string;

pub struct FormatAsJson;

impl<Context> StringFormatter<Context> for FormatAsJson
where
    Context: Serialize,
{
    fn format_string(context: &Context) -> Result<String, Error> {
        Ok(to_string(context)?)
    }
}
```

To allow for error handling, we update the method signature of `format_string` to return a `Result`, with `anyhow::Error` being used as a general error type. We will also be covering better ways to handle errors in a context-generic way in in later chapters.

If we recall from the previous chapter, the `CanFormatIter` trait in fact has the same method signature as `StringFormatter`. So we can refactor the code from the previous chapter, and turn it into a context-generic provider that works for any iterable context like `Vec`.

This use of provider traits can also be more useful in more complex use cases, such as implementing `Serialize`, or even the `Display` trait itself. If we were to implement these traits using CGP, we would also define provider traits such as follows:

```
use core::fmt;
use serde::Serializer;

pub trait ProvideSerialize<Context> {
    fn serialize<S: Serializer>(context: &Context, serializer: S) ->
Result<S::Ok, S::Error>;
}

pub trait ProvideFormat<Context> {
    fn fmt(context: &Context, f: &mut fmt::Formatter<'_>) -> Result<(),
fmt::Error>;
}
```

As we can see above, we can define provider traits for any existing traits by replacing the `Self` type with an explicit `Context` type. In this chapter, we would not be covering the details on how to use CGP and provider traits to simplify formatting and serialization implementations, as that is beyond the current scope. Suffice to say, as we go through later chapters, it will become clearer on how having provider traits can impact us on thinking about how to structure and implement modular code.

Linking Consumers with Providers

In the previous chapter, we learned about how provider traits allow multiple overlapping implementations to be defined. However, if everything is implemented only as provider traits, it would be much more tedious having to determine which provider to use, at every time when we need to use the trait. To overcome this, we would need have *both* provider traits and consumer traits, and have some ways to choose a provider when implementing a consumer trait.

Implementing Consumer Traits

The simplest way to link a consumer trait with a provider is by implementing the consumer trait to call a chosen provider. Consider the `StringFormatter` example of the previous chapter, we would implement `CanFormatString` for a `Person` context as follows:

```
use core::fmt::{self, Display};

pub trait CanFormatString {
    fn format_string(&self) -> String;
}

pub trait StringFormatter<Context> {
    fn format_string(context: &Context) -> String;
}

#[derive(Debug)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

impl Display for Person {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{} {}", self.first_name, self.last_name)
    }
}

impl CanFormatString for Person {
    fn format_string(&self) -> String {
        FormatStringWithDisplay::format_string(self)
    }
}

let person = Person { first_name: "John".into(), last_name: "Smith".into() };

assert_eq!(person.format_string(), "John Smith");
```

To recap the previous chapter, we have a consumer trait `CanFormatString` and a provider trait `StringFormatter`. There are two example providers that implemenent `StringFormatter` - `FormatStringWithDisplay` which formats strings using `Display`, and `FormatStringWithDebug` which formats strings using `Debug`. In addition to that, we implement `CanFormatString` for the `Person` context by forwarding the call to `FormatStringWithDisplay`.

By doing so, we effectively "bind" the `StringFormatter` provider for the `Person` context to `FormatStringWithDisplay`. With that, any time a consumer code calls `person.format_string()`, it would automatically format the context using `Display`.

Thanks to the decoupling of providers and consumers, a context like `Person` can freely choose between multiple providers, and link them with relative ease. Similarly, the provider trait allows multiple context-generic providers such as `FormatStringWithDisplay` and `FormatStringWithDebug` to co-exist.

Blanket Consumer Trait Implementation

In the previous section, we manually implemented `CanFormatString` for `Person` with an explicit call to `FormatStringWithDisplay`. Although the implementation is relatively short, it can become tedious if we make heavy use of provider traits, which would require us to repeat the same pattern for every trait.

To simplify this further, we can make use of *blanket implementations* to automatically delegate the implementation of *all* consumer traits to one chosen provider. We would define the blanket implementation for `CanFormatString` as follows:

```rust
pub trait HasCgpProvider {
    type CgpProvider;
}

pub trait CanFormatString {
    fn format_string(&self) -> String;
}

pub trait StringFormatter<Context> {
    fn format_string(context: &Context) -> String;
}

impl<Context> CanFormatString for Context
where
    Context: HasCgpProvider,
    Context::CgpProvider: StringFormatter<Context>,
{
    fn format_string(&self) -> String {
        Context::CgpProvider::format_string(self)
    }
}
```

First of all, we define a new `HasCgpProvider` trait that contains an associated type `CgpProvider`. The `CgpProvider` type would be specified by a context to choose a provider that it would use to forward all implementations of consumer traits. Following that, we add a blanket implementation for `CanFormatString`, which would be implemented for any `Context` that implements `HasCgpProvider`, provided that `Context::CgpProvider` implements `StringFormatter<Context>`.

To explain in simpler terms - if a context has a provider that implements a provider trait for that context, then the consumer trait for that context is also automatically implemented.

With the new blanket implementation in place, we can now implement `HasCgpProvider` for the `Person` context, and it would now help us to implement `CanFormatString` for free:

```rust
use core::fmt::{self, Display};

#[derive(Debug)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

impl Display for Person {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{} {}", self.first_name, self.last_name)
    }
}

impl HasCgpProvider for Person {
    type CgpProvider = FormatStringWithDisplay;
}

let person = Person { first_name: "John".into(), last_name: "Smith".into() };

assert_eq!(person.format_string(), "John Smith");
```

Compared to before, the implementation of `HasCgpProvider` is much shorter than implementing `CanFormatString` directly, since we only need to specify the provider type without any function definition.

At the moment, because the `Person` context only implements one consumer trait, we can set `FormatStringWithDisplay` directly as `Person::CgpProvider`. However, if there are other consumer traits that we would like to use with `Person`, we would need to define `Person::CgpProvider` with a separate provider that implements multiple provider traits. This will be covered in the next chapter, which we would talk about how to link multiple providers of different provider traits together.

Component System

You may have noticed that the trait for specifying the provider for a context is called `HasCgpProvider` instead of `HasCgpProviders`. This is to generalize the idea of a pair of consumer trait and provider trait working together, forming a *component*.

In context-generic programming, we use the term *component* to refer to a consumer-provider trait pair. The consumer trait and the provider trait are linked together through blanket implementations and traits such as `HasCgpProvider`. These constructs working together to form the basis for a *component system* for CGP.

Provider Delegation

In the previous chapter, we learned to make use of the `HasCgpProvider` trait to define a blanket implementation for a consumer trait like `CanFormatString`, so that a context would automatically delegate the implementation to a provider trait like `StringFormatter`. However, because there can only be one `Components` type defined for `HasCgpProvider`, this means that the given provider needs to implement *all* provider traits that we would like to use for the context.

In this chapter, we will learn to combine multiple providers that each implements a distinct provider trait, and turn them into a single provider that implements multiple provider traits.

Implementing Multiple Providers

Consider that instead of just formatting a context as string, we also want to parse the context from string. In CGP, we would define two separate traits to handle the functionalities separately:

```
use anyhow::Error;

pub trait CanFormatToString {
    fn format_to_string(&self) -> Result<String, Error>;
}

pub trait CanParseFromString: Sized {
    fn parse_from_string(raw: &str) -> Result<Self, Error>;
}
```

Similar to the previous chapter, we define `CanFormatToString` for formatting a context into string, and `CanParseFromString` for parsing a context from a string. Notice that `CanParseFromString` also has an additional `Sized` constraint, as by default the `Self` type in Rust traits do not implement `Sized`, to allow traits to be used in `dyn` trait objects. Compared to before, we also make the methods return a `Result` to handle errors during formatting and parsing. [1] [2]

Next, we also define the provider traits as follows:

```rust
pub trait StringFormatter<Context> {
    fn format_to_string(context: &Context) -> Result<String, Error>;
}

pub trait StringParser<Context> {
    fn parse_from_string(raw: &str) -> Result<Context, Error>;
}

impl<Context> CanFormatToString for Context
where
    Context: HasCgpProvider,
    Context::CgpProvider: StringFormatter<Context>,
{
    fn format_to_string(&self) -> Result<String, Error> {
        Context::CgpProvider::format_to_string(self)
    }
}

impl<Context> CanParseFromString for Context
where
    Context: HasCgpProvider,
    Context::CgpProvider: StringParser<Context>,
{
    fn parse_from_string(raw: &str) -> Result<Context, Error> {
        Context::CgpProvider::parse_from_string(raw)
    }
}
```

Similar to the previous chapter, we make use of blanket implementations and `HasCgpProvider` to link the consumer traits `CanFormatToString` and `CanParseFromString` with their respective provider traits, `StringFormatter` and `StringParser`.

We can then implement context-generic providers for the given provider traits, such as to format and parse the context as JSON if the context implements `Serialize` and `Deserialize`:

```rust
use serde::{Serialize, Deserialize};

pub struct FormatAsJsonString;

impl<Context> StringFormatter<Context> for FormatAsJsonString
where
    Context: Serialize,
{
    fn format_to_string(context: &Context) -> Result<String, Error> {
        Ok(serde_json::to_string(context)?)
    }
}

pub struct ParseFromJsonString;

impl<Context> StringParser<Context> for ParseFromJsonString
where
    Context: for<'a> Deserialize<'a>,
{
    fn parse_from_string(json_str: &str) -> Result<Context, Error> {
        Ok(serde_json::from_str(json_str)?)
    }
}
```

The provider `FormatAsJsonString` implements `StringFormatter` for any `Context` type that implements `Serialize`, and uses `serde_json::to_string` to format the context as JSON. Similarly, the provider `ParseFromJsonString` implements `StringParser` for any `Context` that implements `Deserialize`, and parse the context from a JSON string.

Linking Multiple Providers to a Concrete Context

With the providers implemented, we can now define a concrete context like `Person`, and link it with the given providers. However, since there are multiple providers, we need to first define an *aggregated provider* called `PersonComponents`, which would implement both `StringFormatter` and `StringParser` by delegating the call to the actual providers.

```rust
#[derive(Serialize, Deserialize, Debug, Eq, PartialEq)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

pub struct PersonComponents;

impl HasCgpProvider for Person {
    type CgpProvider = PersonComponents;
}

impl StringFormatter<Person> for PersonComponents {
    fn format_to_string(person: &Person) -> Result<String, Error> {
        FormatAsJsonString::format_to_string(person)
    }
}

impl StringParser<Person> for PersonComponents {
    fn parse_from_string(raw: &str) -> Result<Person, Error> {
        ParseFromJsonString::parse_from_string(raw)
    }
}

let person = Person { first_name: "John".into(), last_name: "Smith".into() };
let person_str = r#"{"first_name":"John","last_name":"Smith"}"#;

assert_eq!(
    person.format_to_string().unwrap(),
    person_str
);

assert_eq!(
    Person::parse_from_string(person_str).unwrap(),
    person
);
```

We first define `Person` struct with auto-derived implementations of `Serialize` and `Deserialize`. We also auto-derive `Debug` and `Eq` for use in tests later on.

We then define a dummy struct `PersonComponents`, which would be used to aggregate the providers for `Person`. Compared to the previous chapter, we implement `HasCgpProvider` for `Person` with `PersonComponents` as the provider.

We then implement the provider traits `StringFormatter` and `StringParser` for `PersonComponents`, with the actual implementation forwarded to `FormatAsJsonString` and `ParseFromJsonString`.

Inside the test that follows, we verify that the wiring indeed automatically implements `CanFormatToString` and `CanParseFromString` for `Person`, with the JSON implementation used.

Blanket Provider Implementation

Although the previous example works, the boilerplate for forwarding multiple implementations by `PersonComponents` seems a bit tedious and redundant. The main differences between the two implementation boilerplate is that we want to choose `FormatAsJsonString` as the provider for `StringFormatter`, and `ParseFromJsonString` as the provider for `StringParser`.

Similar to how we can use `HasCgpProvider` with blanket implementations to link a consumer with a provider, we can reduce the boilerplate required by using similar pattern to link a provider with *another* provider:

```
pub trait DelegateComponent<Name> {
    type Delegate;
}

pub struct StringFormatterComponent;

pub struct StringParserComponent;

impl<Context, Component> StringFormatter<Context> for Component
where
    Component: DelegateComponent<StringFormatterComponent>,
    Component::Delegate: StringFormatter<Context>,
{
    fn format_to_string(context: &Context) -> Result<String, Error> {
        Component::Delegate::format_to_string(context)
    }
}

impl<Context, Component> StringParser<Context> for Component
where
    Component: DelegateComponent<StringParserComponent>,
    Component::Delegate: StringParser<Context>,
{
    fn parse_from_string(raw: &str) -> Result<Context, Error> {
        Component::Delegate::parse_from_string(raw)
    }
}
```

The `DelegateComponent` is similar to the `HasCgpProvider` trait, but it is intended to be implemented by providers instead of concrete contexts. It also has an extra generic `Name` type that is used to differentiate which component the provider delegation is intended for.

To make use of the `Name` parameter, we first need to assign names to the CGP components that we have defined. We first define the dummy struct `StringFormatterComponent` to be used as the name for `StringFormatter`, and `StringParserComponent` to be used as the name for `StringParser`. In general, we can

choose any type as the component name. However by convention, we choose to add a `Component` postfix to the name of the provider trait to be used as the name of the component.

We then define a blanket implementation for `StringFormatter`, which is implemented for a provider type `Component` with the following conditions: if the provider implements `DelegateComponent<StringFormatterComponent>`, and if the associated type `Delegate` also implements `StringFormatter<Context>`, then `Component` also implements `StringFormatter<Context>` by delegating the implementation to `Delegate`.

Following the same pattern, we also define a blanket implementation for `StringParser`. The main difference here is that the name `StringParserComponent` is used as the type argument to `DelegateComponent`. In other words, different blanket provider implementations make use of different `Name` types for `DelegateComponent`, allowing different `Delegate` to be used depending on the `Name`.

Using `DelegateComponent`

It may take a while to fully understand how the blanket implementations with `DelegateComponent` and `HasCgpProvider` work. But since the same pattern will be used everywhere, it would hopefully become clear as we see more examples. It would also help to see how the blanket implementation is used, by going back to the example of implementing the concrete context `Person`.

```rust
#[derive(Serialize, Deserialize, Debug, Eq, PartialEq)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

pub struct PersonComponents;

impl HasCgpProvider for Person {
    type CgpProvider = PersonComponents;
}

impl DelegateComponent<StringFormatterComponent> for PersonComponents {
    type Delegate = FormatAsJsonString;
}

impl DelegateComponent<StringParserComponent> for PersonComponents {
    type Delegate = ParseFromJsonString;
}

let person = Person { first_name: "John".into(), last_name: "Smith".into() };
let person_str = r#"{"first_name":"John","last_name":"Smith"}"#;

assert_eq!(
    person.format_to_string().unwrap(),
    person_str
);

assert_eq!(
    Person::parse_from_string(person_str).unwrap(),
    person
);
```

Instead of implementing the provider traits, we now only need to implement `DelegateComponent<StringFormatterComponent>` and `DelegateComponent<StringParserComponent>` for `PersonComponents`. Rust's trait system would then automatically make use of the blanket implementations to implement `CanFormatToString` and `CanParseFromString` for `Person`.

As we will see in the next chapter, we can make use of macros to further simplify the component delegation, making it as simple as one line to implement such delegation.

Switching Provider Implementations

With the given examples, some readers may question why is there a need to define multiple providers for the JSON implementation, when we can just define one provider struct and implement both provider traits for it.

The use of two providers in this chapter is mainly used as demonstration on how to

delegate and combine multiple providers. In practice, as the number of CGP components increase, we would also quickly run into the need have multiple provider implementations and choosing between different combination of providers.

Even with the simplified example here, we can demonstrate how a different provider for `StringFormatter` may be needed. Supposed that we want to format the context as prettified JSON string, we can define a separate provider `FormatAsPrettifiedJsonString` as follows:

```
pub struct FormatAsPrettifiedJsonString;

impl<Context> StringFormatter<Context> for FormatAsPrettifiedJsonString
where
    Context: Serialize,
{
    fn format_to_string(context: &Context) -> Result<String, Error> {
        Ok(serde_json::to_string_pretty(context)?)
    }
}
```

In the `StringFormatter` implementation for `FormatAsPrettifiedJsonString`, we use `serde_json::to_string_pretty` instead of `serde_json::to_string` to pretty format the JSON string. With CGP, both `FormatAsPrettifiedJsonString` and `FormatAsJsonString` can co-exist peacefully, and we can easily choose which provider to use in each concrete context implementation.

```rust
#[derive(Serialize, Deserialize, Debug, Eq, PartialEq)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

pub struct PersonComponents;

impl HasCgpProvider for Person {
    type CgpProvider = PersonComponents;
}

impl DelegateComponent<StringFormatterComponent> for PersonComponents {
    type Delegate = FormatAsPrettifiedJsonString;
}

impl DelegateComponent<StringParserComponent> for PersonComponents {
    type Delegate = ParseFromJsonString;
}

let person = Person { first_name: "John".into(), last_name: "Smith".into() };
let person_str = r#"{
  "first_name": "John",
  "last_name": "Smith"
}"#;

assert_eq!(
    person.format_to_string().unwrap(),
    person_str,
);

assert_eq!(
    Person::parse_from_string(person_str).unwrap(),
    person
);
```

Compared to before, the only line change is to set the `Delegate` of `DelegateComponent<StringFormatterComponent>` to `FormatAsPrettifiedJsonString` instead of `FormatAsJsonString`. With that, we can now easily choose between whether to pretty print a `Person` context as JSON.

Beyond having a prettified implementation, it is also easy to think of other kinds of generic implementations, such as using `Debug` or `Display` to format strings, or use different encodings such as XML to format the string. With CGP, we can define generalized component interfaces that are applicable to a wide range of implementations. We can then make use of `DelegateComponent` to easily choose which implementation we want to use for different concrete contexts.

---

1. A proper introduction to error handling using CGP will be covered in future chapters. But for now, we will use use `anyhow::Error` to handle errors in a more naive way. ↩

2. There are more general terms the problem space of formatting and parsing a context, such as *serialization* or *encoding*. Instead of strings, a more general solution may use types such as *bytes* or *buffers*. Although it is possible to design a generalized solution for encoding in CGP, it would be too much to cover the topic in this chapter alone. As such, we use naive strings in this chapter so that we can focus on first understanding the basic concepts in CGP. ↩

Debugging Support

By leveraging impl-side dependencies, CGP providers are able to include additional dependencies that are not specified in the provider trait. We have already seen this in action in the previous chapter, for example, where the provider `FormatAsJsonString` is able to require `Context` to implement `Serialize`, while that is not specified anywhere in the provider trait `StringFormatter`.

We have also went through how provider delegation can be done using `DelegateComponent`, which an aggregated provider like `PersonComponents` can use to delegate the implementation of `StringFormatter` to `FormatAsJsonString`. Within this delegation, we can also see that the requirement for `Context` to implement `Serialize` is not required in any part of the code.

In fact, because the provider constraints are not enforced in `DelegateComponent`, the delegation would always be successful, even if some provider constraints are not satisfied. In other words, the impl-side provider constraints are enforced *lazily* in CGP, and compile-time errors would only arise when we try to use a consumer trait against a concrete context.

Unsatisfied Dependency Errors

To demonstrate how such error would arise, we would reuse the same example `Person` context as the previous chapter. Consider if we made a mistake and forgot to implement `Serialize` for `Person`:

```rust
// Note: We pretend to forgot to derive Serialize here
#[derive(Deserialize, Debug, Eq, PartialEq)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

pub struct PersonComponents;

impl HasCgpProvider for Person {
    type CgpProvider = PersonComponents;
}

impl DelegateComponent<StringFormatterComponent> for PersonComponents {
    type Delegate = FormatAsJsonString;
}

impl DelegateComponent<StringParserComponent> for PersonComponents {
    type Delegate = ParseFromJsonString;
}
```

We know that `Person` uses `PersonComponents` to implement `CanFormatToString`, and `PersonComponents` delegates the provider implementation to `FormatAsJsonString`. However, since `FormatAsJsonString` requires `Person` to implement `Serialize`, without it `CanFormatToString` cannot be implemented on `PersonContext`.

However, notice that the above code still compiles successfully. This is because we have not yet try to use `CanFormatToString` on person. We can try to add test code to call `format_to_string`, and check if it works:

```rust
let person = Person { first_name: "John".into(), last_name: "Smith".into() };
println!("{}", person.format_to_string().unwrap());
```

The first time we try to call the method, our code would fail with a compile error that looks like follows:

```
error[E0599]: the method `format_to_string` exists for struct `Person`, but
its trait bounds were not satisfied
  --> debugging-techniques.md:180:23
   |
54 | pub struct Person {
   | ----------------- method `format_to_string` not found for this struct
because it doesn't satisfy `Person: CanFormatToString`
...
59 | pub struct PersonComponents;
   | --------------------------- doesn't satisfy `PersonComponents:
StringFormatter<Person>`
...
73 | println!("{}", person.format_to_string().unwrap());
   |                -------^^^^^^^^^^^^^^^^--
   |                |      |
   |                |      this is an associated function, not a method
   |                help: use associated function syntax instead:
`Person::format_to_string()`
   |
   = note: found the following associated functions; to be used as methods,
functions must have a `self` parameter
note: the candidate is defined in the trait `StringFormatter`
  --> debugging-techniques.md:125:5
   |
18 |     fn format_to_string(&self) -> Result<String, Error>;
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
note: trait bound `PersonComponents: StringFormatter<Person>` was not
satisfied
  --> debugging-techniques.md:119:1
   |
12 | / #[cgp_component {
13 | |    name: StringFormatterComponent,
14 | |    provider: StringFormatter,
15 | |    context: Context,
   | |             ^^^^^^^
16 | | }]
   | |__^
note: the trait `StringFormatter` must be implemented
  --> debugging-techniques.md:124:1
   |
17 | / pub trait CanFormatToString {
18 | |     fn format_to_string(&self) -> Result<String, Error>;
19 | | }
   | |_^
   = help: items from traits can only be used if the trait is implemented and
in scope
note: `CanFormatToString` defines an item `format_to_string`, perhaps you
need to implement it
  --> debugging-techniques.md:124:1
   |
17 | pub trait CanFormatToString {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^
   = note: this error originates in the attribute macro `cgp_component` (in
Nightly builds, run with -Z macro-backtrace for more info)

error: aborting due to 1 previous error
```

Unfortunately, the error message returned from Rust is very confusing, and not helpful at all in guiding us to the root cause. For an inexperience developer, the main takeaway from the error message is just that `CanFormatString` is not implemented for `Person`, but the developer is left entirely on their own to find out how to fix it.

One main reason we get such obscured errors is because the implementation of `CanFormatString` is done through two indirect blanket implementations. As Rust was not originally designed for blanket implementations to be used this way, it does not follow through to explain why the blanket implementation is not implemented.

Technically, there is no reason why the Rust compiler cannot be improved to show more detailed errors to make using CGP easier. However, improving the compiler will take time, and we need to present strong argument on why such improvement is needed, e.g. through this book. Until then, we need workarounds to make it easier to debug CGP errors in the meanwhile.

`IsProviderFor` Trait

Since `cgp` v0.4, we have developed a technique to explicitly propagate the constraint requirements, and "trick" Rust to show us the error messages that we need. We will first define a `IsProviderFor` trait as follows:

```
pub trait IsProviderFor<Component, Context, Params = ()> {}
```

The `IsProviderFor` trait is a marker trait that can be trivially implemented. It is intended to be implemented by provider structs, and is parameterized by 3 generic parameters:

- `Component` - The component name type that corresponds to the provider implementation.
- `Context` - The context type that is implemented by the provider.
- `Params` - Any additional generic parameters present in the provider trait, combined as a tuple.

Even though the `IsProviderFor` trait can be trivially implemented, we intentionally include additional constraints that are exactly the same as the original constraints specified in the corresponding provider trait implementation.

For example, the provider `FormatAsJsonString` in the earlier example would implement `IsProviderFor` as follows:

```rust
pub struct FormatAsJsonString;

impl<Context> StringFormatter<Context> for FormatAsJsonString
where
    Context: Serialize,
{
    fn format_to_string(context: &Context) -> Result<String, Error> {
        Ok(serde_json::to_string(context)?)
    }
}

impl<Context> IsProviderFor<StringFormatterComponent, Context> for
FormatAsJsonString
where
    Context: Serialize,
{ }
```

The way to understand the trait implementation is follows: `FormatAsJsonString` has a provider implementation for `StringFormatterComponent` with the context `Context`, given that `Context: Serialize`.

We can think of `IsProviderFor` trait to act as a "carrier" for the hidden constraints of provider traits. With it, instead of trying to check each provider trait implementation, we only need to check for the implementation of one trait, `IsProviderFor`.

Propagating `IsProviderFor` Constraints

Now that we have captured our constraints using `IsProviderFor`, we need to somehow propagate the constraint upward through the delegation chain, so that the context provider `PersonComponents` also implements `IsProviderFor` with the same constraints as its delegate, `FormatAsJsonString`.

We would do that by modifying the provider trait definition, so that `IsProviderFor` becomes a *supertrait* of the provider trait:

```
pub trait StringFormatter<Context>: IsProviderFor<StringFormatterComponent,
Context> {
    fn format_to_string(context: &Context) -> Result<String, Error>;
}

impl<Context, Component> StringFormatter<Context> for Component
where
    Component: DelegateComponent<StringFormatterComponent>
      + IsProviderFor<StringFormatterComponent, Context>,
    Component::Delegate: StringFormatter<Context>,
{
    fn format_to_string(context: &Context) -> Result<String, Error> {
        Component::Delegate::format_to_string(context)
    }
}
```

We make it a requirement that in order for a provider to implement `StringFormatter<Context>`, the provider also needs to implement `IsProviderFor<StringFormatterComponent, Context>`. The supertrait constraint not only makes sure that we don't forget to always implement `IsProviderFor`, but also triggers Rust to show any unsatisfied constraints that were hidden previously.

Additionally, we also modify the blanket implementation of `StringFormatter`, so that when a provider like `PersonComponents` delegates the implementation, it would need to explicitly implement `IsProviderFor<StringFormatterComponent, Context>` in addition to implementing `DelegateComponent<StringFormatterComponent>`.

With the new requirements in place, when we delegate `PersonComponents`'s provider implementation of `StringFormatterComponent`, we would also write an implementation of `IsProviderFor` for it as follows:

```
impl DelegateComponent<StringFormatterComponent> for PersonComponents {
    type Delegate = FormatAsJsonString;
}

impl<Context> IsProviderFor<StringFormatterComponent, Context>
   for PersonComponents
where
    FormatAsJsonString: IsProviderFor<StringFormatterComponent, Context>
{
}
```

Notice here that when implementing `IsProviderFor` for `PersonComponents`, we add a constraint that directly requires `FormatAsJsonString` to also implement `IsProviderFor` with the same generic parameters. By doing so, we resurface the constraints to Rust, so that it would recursively look into the `IsProviderFor` trait bounds, and print out any unsatisfied constraints in the error messages.

At this point, you may wonder why not link the provider trait directly within the delegated

implementation of `IsProviderFor`, such as:

```
impl<Context> IsProviderFor<StringFormatterComponent, Context>
   for PersonComponents
where
    FormatAsJsonString: StringFormatter<Context>,
{
}
```

The main reason to *not* do this is that it requires direct access to the provider trait, which is not as simple dealing with simple types. Furthermore, the provider trait may contain additional where clauses, which would also need to be propagated explicitly.

By making use of `IsProviderFor`, we are essentially "erasing" everything at the trait level, and use a single trait to represent all other provider traits. After all, the only thing that we are interested here is to propagate the constraints for the purpose of showing better error messages.

Check Traits

Now that we have the wirings for `IsProviderFor` in place, we can implement *check traits* to check on whether the provider traits that we want to use with `Person` are implemented by its provider, `PersonComponents`.

```
pub trait CanUsePersonComponents:
    IsProviderFor<StringFormatterComponent, Person>
    + IsProviderFor<StringParserComponent, Person>
{
}

impl CanUsePersonComponents for PersonComponents {}
```

To put it simply, a check trait is defined just for checking whether a type implements other traits that we are interested to use with that type. For our case, we want to check that `PersonComponents: IsProviderFor<StringFormatterComponent, Person>` and `IsProviderFor<StringParserComponent, Person>`, since we expect `PersonComponents` to implement `StringFormatter<Person>` and `StringParser<Person>`. So we define a check trait called `CanUsePersonComponents`, and put the `IsProviderFor` constraints as the supertrait.

The check trait is then followed by an implementation of `CanUsePersonComponents` for `PersonComponents`, which acts as an *assertion* that `PersonComponents` implements all the supertraits that we specified.

With the checks in place, we now get a compile error message that shows us exactly what

we need to fix, which is to implement `Serialize` for `Person`:

```
error[E0277]: the trait bound `Person: Serialize` is not satisfied
   --> src/lib.rs:155:33
    |
155 | impl CanUsePersonComponents for PersonComponents {}
    |                                 ^^^^^^^^^^^^^^^^ the trait `Serialize`
is not implemented for `Person`
    |
    = note: for local types consider adding `#[derive(serde::Serialize)]` to
your `Person` type
    = note: for types from other crates check whether the crate offers a
`serde` feature flag
    = help: the following other types implement trait `Serialize`:
              &'a T
              &'a mut T
              ()
              (T,)
              (T0, T1)
              (T0, T1, T2)
              (T0, T1, T2, T3)
              (T0, T1, T2, T3, T4)
            and 131 others
note: required for `FormatAsJsonString` to implement
`IsProviderFor<StringFormatterComponent, Person>`
   --> src/lib.rs:91:15
    |
91  | impl<Context> IsProviderFor<StringFormatterComponent, Context>
    |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
92  |     for FormatAsJsonString
    |         ^^^^^^^^^^^^^^^^^^
93  | where
94  |     Context: Serialize,
    |              --------- unsatisfied trait bound introduced here
    = note: 1 redundant requirement hidden
    = note: required for `PersonComponents` to implement
`IsProviderFor<StringFormatterComponent, Person>`
note: required by a bound in `CanUsePersonComponents`
   --> src/lib.rs:150:5
    |
149 | pub trait CanUsePersonComponents:
    |           ---------------------- required by a bound in this trait
150 |     IsProviderFor<StringFormatterComponent, Person>
    |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ required by this
bound in `CanUsePersonComponents`
```

As we can see, the use of `IsProviderFor` helps make it possible for us to debug our CGP programs again.

`CanUseComponent` Trait

Although we can directly check for the implementation of `IsProviderFor`, the definition is not as straightforward as defining checks directly with the `Person` context. Furthermore, the context type `Person` needs to be repeatedly specified for each supertrait, which becomes tedious and adds noise.

We can improve the check definitions by introducing a helper `CanUseComponent` trait that is defined as follows:

```
pub trait IsProviderFor<Component, Context, Params = ()> {}

pub trait CanUseComponent<Component, Params = ()> {}

impl<Context, Component, Params> CanUseComponent<Component, Params> for
Context
where
    Context: HasCgpProvider,
    Context::CgpProvider: IsProviderFor<Component, Context, Params>,
{
}
```

Instead of being implemented by a provider, `CanUseComponent` is intended to be implemented by a context type that implements `HasCgpProvider`. Compared to `IsProviderFor`, there is no need to specify an explicit `Context` parameter.

`CanUseComponent` also has a blanket implementation, making it automatically implemented for any `Context` type that implements `HasCgpProvider`, with `Context::CgpProvider` implementing `IsProviderFor`.

In other words, we are using `CanUseComponent` as a *trait alias* to `IsProviderFor`, with improved ergonomics that we can check directly against a context type.

Using `CanUseComponent`, we can re-define our check trait much simply as follows:

```
pub trait CanUsePerson:
    CanUseComponent<StringFormatterComponent>
    + CanUseComponent<StringParserComponent>
{
}

impl CanUsePerson for Person {}
```

Compared to before, we define the check trait `CanUsePerson` directly on `Person` instead of `PersonComponents`. When we try compiling, we would still get the same error message that shows us the required information to fix the error:

```
error[E0277]: the trait bound `Person:
CanUseComponent<StringFormatterComponent>` is not satisfied
   --> src/lib.rs:164:23
    |
164 | impl CanUsePerson for Person {}
    |                       ^^^^^^ the trait `Serialize` is not implemented
for `Person`
...
```

Limitations

The use of `IsProviderFor` significantly improves the developer experience for CGP, which was significantly harder to debug prior to the introduction of this technique in v0.4.0.

At this point, you might be concerned of the additional boilerplate required to implement and propagate the constraints for `IsProviderFor`. However, as we will see in the next chapter, the CGP macros will automate the bulk of the implementation of `IsProviderFor`, and only require lightweight attribute macros to be applied to enable the code generation.

It is worth noting that the `IsProviderFor` trait is introduced as a workaround to improve the error messages of CGP code in Rust. Hypothetically, if Rust could provide better support of showing the relevant error messages, we could entirely remove the use of `IsProviderFor` in future versions of CGP.

On the other hand, the use of `IsProviderFor` can serve as a great showcase to the Rust compiler team on what error messages should have been shown by default, and make it easier to evaluate what should be the official fix in the Rust compiler.

That said, the error messages shown via `IsProviderFor` can sometimes be too verbose, especially when an application contains many providers with deeply nested dependencies. The reason is that any unsatisfied constraint from deeply nested dependencies can propagate all the way up through chains of `IsProviderFor` implementations. Although this can help us pinpoint the root cause, it could generate too much noise when showing all errors from the intermediary layers.

When encountering heap of error messages generated from `IsProviderFor`, a useful tip is that the relevant error message may be hiding near the bottom. So it may be useful to read from the bottom up instead of top down.

Interactive Debugging with Argus

A potential improvement that we are currently exploring is to make use of Argus to help navigate error messages generated from CGP code. Argus provides an interactive debugger for trait-related errors, which may be well suited to be used for debugging CGP code.

We will add further details in the future to share potential integration with Argus. For now, interested readers are encouraged to check out the project, and perhaps make contribution to make such integration possible.

Conclusion

In this chapter, we have learned how CGP makes use of the `IsProviderFor` trait to help show relevant error messages when encountering unsatisfied constraints. In the next chapter, we will walk through how to automate the generation of all the relevant boilerplates that we have learned so far, and write succint CGP code using macros.

We will show the full example that we have walked through earlier, with the addition of `IsProviderFor` into the code:

```rust
use anyhow::Error;
use serde::{Deserialize, Serialize};

pub trait HasCgpProvider {
    type CgpProvider;
}

pub trait IsProviderFor<Component, Context, Params = ()> {}

pub trait DelegateComponent<Name> {
    type Delegate;
}

pub trait CanUseComponent<Component, Params = ()> {}

impl<Context, Component, Params> CanUseComponent<Component, Params> for
Context
where
    Context: HasCgpProvider,
    Context::CgpProvider: IsProviderFor<Component, Context, Params>,
{
}

pub struct StringFormatterComponent;

pub struct StringParserComponent;

pub trait CanFormatToString {
    fn format_to_string(&self) -> Result<String, Error>;
}

pub trait CanParseFromString: Sized {
    fn parse_from_string(raw: &str) -> Result<Self, Error>;
}

pub trait StringFormatter<Context>:
    IsProviderFor<StringFormatterComponent, Context>
{
    fn format_to_string(context: &Context) -> Result<String, Error>;
}

pub trait StringParser<Context>:
    IsProviderFor<StringParserComponent, Context>
{
    fn parse_from_string(raw: &str) -> Result<Context, Error>;
}

impl<Context> CanFormatToString for Context
where
    Context: HasCgpProvider,
    Context::CgpProvider: StringFormatter<Context>,
{
    fn format_to_string(&self) -> Result<String, Error> {
        Context::CgpProvider::format_to_string(self)
    }
}
```

```rust
impl<Context> CanParseFromString for Context
where
    Context: HasCgpProvider,
    Context::CgpProvider: StringParser<Context>,
{
    fn parse_from_string(raw: &str) -> Result<Context, Error> {
        Context::CgpProvider::parse_from_string(raw)
    }
}

impl<Context, Component> StringFormatter<Context> for Component
where
    Component: DelegateComponent<StringFormatterComponent>
        + IsProviderFor<StringFormatterComponent, Context>,
    Component::Delegate: StringFormatter<Context>,
{
    fn format_to_string(context: &Context) -> Result<String, Error> {
        Component::Delegate::format_to_string(context)
    }
}

impl<Context, Component> StringParser<Context> for Component
where
    Component: DelegateComponent<StringParserComponent>
        + IsProviderFor<StringParserComponent, Context>,
    Component::Delegate: StringParser<Context>,
{
    fn parse_from_string(raw: &str) -> Result<Context, Error> {
        Component::Delegate::parse_from_string(raw)
    }
}

pub struct FormatAsJsonString;

impl<Context> StringFormatter<Context> for FormatAsJsonString
where
    Context: Serialize,
{
    fn format_to_string(context: &Context) -> Result<String, Error> {
        Ok(serde_json::to_string(context)?)
    }
}

impl<Context> IsProviderFor<StringFormatterComponent, Context>
    for FormatAsJsonString
where
    Context: Serialize,
{
}

pub struct ParseFromJsonString;

impl<Context> StringParser<Context> for ParseFromJsonString
where
    Context: for<'a> Deserialize<'a>,
{
```

```
    fn parse_from_string(json_str: &str) -> Result<Context, Error> {
        Ok(serde_json::from_str(json_str)?)
    }
}

impl<Context> IsProviderFor<StringParserComponent, Context>
    for ParseFromJsonString
where
    Context: for<'a> Deserialize<'a>,
{
}

// Note: We pretend to forgot to derive Serialize here
#[derive(Deserialize, Debug, Eq, PartialEq)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

pub struct PersonComponents;

impl HasCgpProvider for Person {
    type CgpProvider = PersonComponents;
}

impl DelegateComponent<StringFormatterComponent> for PersonComponents {
    type Delegate = FormatAsJsonString;
}

impl<Context> IsProviderFor<StringFormatterComponent, Context>
    for PersonComponents
where
    FormatAsJsonString: IsProviderFor<StringFormatterComponent, Context>,
{
}

impl DelegateComponent<StringParserComponent> for PersonComponents {
    type Delegate = ParseFromJsonString;
}

impl<Context> IsProviderFor<StringParserComponent, Context> for
PersonComponents where
    ParseFromJsonString: IsProviderFor<StringParserComponent, Context>
{
}

pub trait CanUsePerson:
    CanUseComponent<StringFormatterComponent>
    + CanUseComponent<StringParserComponent>
{
}

impl CanUsePerson for Person {}
```

Component Macros

At this point, we have covered all basic building blocks of defining CGP components. In summary, a CGP component is consist of the following building blocks:

- A consumer trait.
- A provider trait.
- A component name type.
- A blanket implementation of the consumer trait using `HasCgpProvider`.
- A blanket implementation of the provider trait using `DelegateComponent`.

Syntactically, all CGP components follow the same pattern. The pattern is roughly as follows:

```rust
// Component name
pub struct ActionPerformerComponent;

// Consumer trait
pub trait CanPerformAction<GenericA, GenericB, ...>:
    ConstraintA + ConstraintB + ...
{
    fn perform_action(
        &self,
        arg_a: ArgA,
        arg_b: ArgB,
        ...
    ) -> Output;
}

// Provider trait
pub trait ActionPerformer<Context, GenericA, GenericB, ...>:
    IsProviderFor<ActionPerformerComponent, Context, (GenericA, GenericB,
...)>
where
    Context: ConstraintA + ConstraintB + ...,
{
    fn perform_action(
        context: &Context,
        arg_a: ArgA,
        arg_b: ArgB,
        ...
    ) -> Output;
}

// Blanket implementation for consumer trait
impl<Context, GenericA, GenericB, ...>
    CanPerformAction<GenericA, GenericB, ...> for Context
where
    Context: HasCgpProvider + ConstraintA + ConstraintB + ...,
    Context::Components: ActionPerformer<Context>,
{
    fn perform_action(
        &self,
        arg_a: ArgA,
        arg_b: ArgB,
        ...
    ) -> Output {
        Context::Components::perform_action(self, arg_a, arg_b, ...)
    }
}

// Blanket implementation for provider trait
impl<Context, Component, GenericA, GenericB, ...>
    ActionPerformer<Context, GenericA, GenericB, ...>
    for Component
where
    Context: ConstraintA + ConstraintB + ...,
    Component: DelegateComponent<ActionPerformerComponent>
        + IsProviderFor<ActionPerformerComponent, Context, (GenericA,
GenericB, ...)>,
```

```
    Component::Delegate: ActionPerformer<Context, GenericA, GenericB, ...>,
{
    fn perform_action(
        context: &Context,
        arg_a: ArgA,
        arg_b: ArgB,
        ...
    ) -> Output {
        Component::Delegate::perform_action(context, arg_a, arg_b, ...)
    }
}
```

`#[cgp_component]` Macro

With the repetitive pattern, it makes sense that we should be able to just define the consumer trait, and make use of Rust macros to generate the remaining code. The author has published the `cgp` Rust crate that provides the `cgp_component` attribute macro that can be used for this purpose. Using the macro, the same code as above can be significantly simplified to the following:

```
use cgp::prelude::*;

#[cgp_component {
    name: ActionPerformerComponent,
    provider: ActionPerformer,
    context: Context,
}]
pub trait CanPerformAction<GenericA, GenericB, ...>:
    ConstraintA + ConstraintB + ...
{
    fn perform_action(
        &self,
        arg_a: ArgA,
        arg_b: ArgB,
        ...
    ) -> Output;
}
```

To use the macro, the bulk import statement `use cgp::prelude::*` has to be used to bring all CGP constructs into scope. This includes the `HasCgpProvider` and `DelegateComponent` traits, which are also provided by the `cgp` crate.

We then use `cgp_component` as an attribute proc macro, with several key-value arguments given. The `name` field is used to define the component name type, which is called `ActionPerformerComponent`. The `provider` field `ActionPerformer` is used for the name for the provider trait. The `context` field `Context` is used for the generic type name of the context when used inside the provider trait.

The `cgp_component` macro allows the `name` and `context` field to be omited. When omitted, the `context` field will default to `Context`, and the `name` field will default to `{provider}Component`. So the same example above could be simplified to:

```
#[cgp_component {
    provider: ActionPerformer,
}]
pub trait CanPerformAction<GenericA, GenericB, ...>:
    ConstraintA + ConstraintB + ...
{
    fn perform_action(
        &self,
        arg_a: ArgA,
        arg_b: ArgB,
        ...
    ) -> Output;
}
```

When only the provider name is specified, we can also omit the `key: value` syntax, and specify only the provider name:

```
#[cgp_component(ActionPerformer)]
pub trait CanPerformAction<GenericA, GenericB, ...>:
    ConstraintA + ConstraintB + ...
{
    fn perform_action(
        &self,
        arg_a: ArgA,
        arg_b: ArgB,
        ...
    ) -> Output;
}
```

`delegate_components!` Macro

In addition to the `cgp_component` macro, `cgp` also provides the `delegate_components!` macro that can be used to automatically implement `DelegateComponent` for a provider type. The syntax is roughly as follows:

```
use cgp::prelude::*;

pub struct TargetProvider;

delegate_components! {
    TargetProvider {
        ComponentA: ProviderA,
        ComponentB: ProviderB,
        [
            ComponentC1,
            ComponentC2,
            ...
        ]: ProviderC,
    }
}
```

The above code would be desugared into the following:

```
impl DelegateComponent<ComponentA> for TargetProvider {
    type Delegate = ProviderA;
}

impl<Context, Params> IsProviderFor<ComponentA, Context, Params>
    for TargetProvider
where
    ProviderA: IsProviderFor<ComponentA, Context, Params>,
{
}

impl DelegateComponent<ComponentB> for TargetProvider {
    type Delegate = ProviderB;
}

impl<Context, Params> IsProviderFor<ComponentB, Context, Params>
    for TargetProvider
where
    ProviderB: IsProviderFor<ComponentB, Context, Params>,
{
}

impl DelegateComponent<ComponentC1> for TargetProvider {
    type Delegate = ProviderC;
}

impl<Context, Params> IsProviderFor<ComponentC1, Context, Params>
    for TargetProvider
where
    ProviderC: IsProviderFor<ComponentC1, Context, Params>,
{
}

impl DelegateComponent<ComponentC2> for TargetProvider {
    type Delegate = ProviderC;
}

impl<Context, Params> IsProviderFor<ComponentC2, Context, Params>
    for TargetProvider
where
    ProviderC: IsProviderFor<ComponentC2, Context, Params>,
{
}
```

The `delegate_components!` macro accepts an argument to an existing type, `TargetProvider`, which is expected to be defined outside of the macro. It is followed by an open brace, and contain entries that look like key-value pairs.

For a key-value pair `ComponentA: ProviderA`, the type `ComponentA` is used as the component name, and `ProviderA` refers to the provider implementation. When multiple keys map to the same value, i.e. multiple components are delegated to the same provider implementation, the array syntax can be used to further simplify the mapping.

Instead of defining the provider struct on our own, we can also instruct `delegate_components!` to also define the provider struct for us, by adding a `new` keyword in front:

```
delegate_components! {
    new TargetProvider {
        ComponentA: ProviderA,
        ComponentB: ProviderB,
        [
            ComponentC1,
            ComponentC2,
            ...
        ]: ProviderC,
    }
}
```

`#[cgp_context]` Macro

The `#[cgp_context]` macro can be applied on a context struct, to automatically define the provider struct for the context and implement `HasCgpProvider` for the context.

Given the following context definition:

```
#[cgp_context(MyContextComponents)]
pub struct MyContext {
    ...
}
```

The macro will generate the following constructs:

```
pub struct MyContextComponents;

impl HasCgpProvider for MyContext {
    type CgpProvider = MyContextComponents;
}
```

If the context provider name follows the pattern `{ContextName}Components`, then the macro attribute argument can be omitted, and the code can be simplified to:

```
#[cgp_context]
pub struct MyContext {
    ...
}
```

`#[cgp_provider]` Macro

When implementing a provider, the `#[cgp_provider]` macro needs to be used to automatically implement the `IsProviderFor` implementation, with all constraints within the `impl` block copied over.

Given a provider trait implementation with the pattern:

```
pub struct Provider;

#[cgp_provider(ActionPerformerComponent)]
impl<Context, GenericA, GenericB, ...>
    ActionPerformer<Context, GenericA, GenericB, ...>
    for Provider
where
    Context: ConstraintA + ConstraintB + ...,
    Context::Assoc: ConstraintC + ConstraintD + ...,
{ ... }
```

`#[cgp_provider]` would generate an `IsProviderFor` implementation that follows the pattern:

```
impl<Context, GenericA, GenericB, ...>
    IsProviderFor<ActionPerformerComponent, Context, GenericA, GenericB, ...>
    for Provider
where
    Context: ConstraintA + ConstraintB + ...,
    Context::Assoc: ConstraintC + ConstraintD + ...,
{ }
```

If the component name for the provider trait follows the format `"{ProviderTraitName}Component"`, then the component name can be omitted in the attribute argument for `#[cgp_provider]`, simplifying the code to:

```
pub struct Provider;

#[cgp_provider]
impl<Context, GenericA, GenericB, ...>
    ActionPerformer<Context, GenericA, GenericB, ...>
    for Provider
where
    Context: ConstraintA + ConstraintB + ...,
    Context::Assoc: ConstraintC + ConstraintD + ...,
{ ... }
```

Note, however, that the generated code would require the component type `"{ProviderTraitName}Component"` to be imported into scope. If the component name is not specified, IDEs like Rust Analyzer may not provide quick fix for auto importing the component. As a result, it may still be preferrable to include the component name

attribute, especially when writing new code.

There is also a variant of the macro, `#[cgp_new_provider]`, which would also automatically define the struct for the provider. With that, the code can be defined with the `struct` definition omitted:

```
#[cgp_new_provider]
impl<Context, GenericA, GenericB, ...>
    ActionPerformer<Context, GenericA, GenericB, ...>
    for Provider
where
    Context: ConstraintA + ConstraintB + ...,
    Context::Assoc: ConstraintC + ConstraintD + ...,
{ ... }
```

`#[cgp_new_provider]` is mainly useful in cases where a provider only implements one provider trait. When defininining a provider with multiple provider trait implementations, it may be more clear to define the provider struct explicitly, or only use `#[cgp_new_provider]` for the first `impl` block of the provider.

`check_components!` Macro

To help with debugging CGP code, the `check_components!` macro is provided to allow us to quickly write compile-time tests on the component wiring.

Given the following code pattern:

```
check_components! {
    CanUseContext for Context {
        ComponentA,
        ComponentB,
        ComponentC: GenericA,
        [
            ComponentD,
            ComponentE,
        ]: [
            (GenericB1, GenericB2, ...),
            (GenericC1, GenericC2, ...),
        ],
    }
}
```

The following check trait would be generated:

```
pub trait CanUseContext:
    CanUseComponent<ComponentA>
    + CanUseComponent<ComponentB>
    + CanUseComponent<ComponentC, GenericA>
    + CanUseComponent<ComponentD, (GenericB1, GenericB2, ...)>
    + CanUseComponent<ComponentD, (GenericC1, GenericC2, ...)>
    + CanUseComponent<ComponentE, (GenericB1, GenericB2, ...)>
    + CanUseComponent<ComponentE, (GenericC1, GenericC2, ...)>
{}

impl CanUseContext for Context {}
```

The `check_components!` macro allows the use of array syntax at either the key or value position, when there are multiple components that share the same set of generic parameters.

`delegate_and_check_components!` Macro

The `delegate_and_check_components!` macro combines both calls to `delegate_components!` and `check_components!`, so that wiring checks are done as soon as a delegate entry is added. This can simplify the boilerplate required to duplicate the code of listing all components in both delegate and check entries.

Given the following:

```
delegate_and_check_components! {
    CanUseContext for Context;
    ContextComponents {
        ComponentA: ProviderA,
        ComponentB: ProviderB,
        [
            ComponentC1,
            ComponentC2,
            ...
        ]: ProviderC,
    }
}
```

The macro would expand into the equivalent of:

```
delegate_components! {
    ContextComponents {
        ComponentA: ProviderA,
        ComponentB: ProviderB,
        [
            ComponentC1,
            ComponentC2,
            ...
        ]: ProviderC,
    }
}

check_components! {
    CanUseContext for Context {
        ComponentA,
        ComponentB,
        ComponentC1,
        ComponentC2,
    }
}
```

You may wonder why we need define a separate macro, instead of always checking the wiring directly inside `delegate_components!`. The main reason is that while `delegate_and_check_components!` can work for the simple cases, it is more limited and cannot handle well on advanced cases where the CGP traits contain additional generic parameters. For such cases, it is still better to call `delegate_components!` and `check_components!` separately.

Example Use

To illustrate how `cgp_component` and `delegate_components` can be used, we revisit the code for `CanFormatToString`, `CanParseFromString`, and `PersonContext` from the previous chapter, and look at how the macros can simplify the same code.

Following is the full code after simplification using `cgp`:

```rust
use cgp::prelude::*;
use anyhow::Error;
use serde::{Serialize, Deserialize};

// Component definitions

#[cgp_component(StringFormatter)]
pub trait CanFormatToString {
    fn format_to_string(&self) -> Result<String, Error>;
}

#[cgp_component(StringParser)]
pub trait CanParseFromString: Sized {
    fn parse_from_string(raw: &str) -> Result<Self, Error>;
}

// Provider implementations

#[cgp_new_provider]
impl<Context> StringFormatter<Context> for FormatAsJsonString
where
    Context: Serialize,
{
    fn format_to_string(context: &Context) -> Result<String, Error> {
        Ok(serde_json::to_string(context)?)
    }
}

#[cgp_new_provider]
impl<Context> StringParser<Context> for ParseFromJsonString
where
    Context: for<'a> Deserialize<'a>,
{
    fn parse_from_string(json_str: &str) -> Result<Context, Error> {
        Ok(serde_json::from_str(json_str)?)
    }
}

// Concrete context and wiring
#[cgp_context]
#[derive(Serialize, Deserialize, Debug, Eq, PartialEq)]
pub struct Person {
    pub first_name: String,
    pub last_name: String,
}

delegate_and_check_components! {
    CanUsePerson for Person;
    PersonComponents {
        StringFormatterComponent:
            FormatAsJsonString,
        StringParserComponent:
            ParseFromJsonString,
    }
}
```

As we can see, the new code is significantly simpler and more readable than before. Using `#[cgp_component]`, we no longer need to explicitly define the provider traits `StringFormatter` and `StringParser`, and the blanket implementations can be omitted.

With `#[cgp_new_provider]`, the `IsProviderFor` implementations for `FormatAsJsonString` and `ParseFromJsonString` are automatically implemented, together with the struct definitions.

We also make use of `delegate_and_check_components!` on `PersonComponents` to delegate `StringFormatterComponent` to `FormatAsJsonString`, and `StringParserComponent` to `ParseFromJsonString`, and then check to ensure that the wirings are implemented correctly for the `Person` context.

CGP Macros as Language Extension

The use of `cgp` crate with its macros is essential in enabling the full power of context-generic programming in Rust. Without it, programming with CGP would become too verbose and full of boilerplate code.

On the other hand, the use of `cgp` macros makes CGP code look much more like programming in a *domain-specific language* (DSL) than in regular Rust. In fact, one could argue that CGP acts as a *language extension* to the base language Rust, and almost turn into its own programming language.

In a way, implementing CGP in Rust is slightly similar to implementing OOP in C. We could think of context-generic programming being as foundational as object-oriented programming, and may be integrated as a core language feature in future programming languages.

Perhaps one day, there might be an equivalent of C++ to replace CGP-on-Rust. Or perhaps more ideally, the core constructs of CGP would one day directly supported as a core language feature in Rust. But until that happens, the `cgp` crate serves as an experimental ground on how context-generic programming can be done in Rust, and how it can help build better Rust applications.

In the chapters that follow, we will make heavy use of `cgp` and its macros to dive further into the world of context-generic programming.

Associated Types

In the first part of this book, we explored how CGP leverages Rust's trait system to wire up components using blanket implementations. Because CGP operates within Rust's trait system, it allows us to incorporate advanced Rust features to create new design patterns. In this chapter, we will focus on using *associated types* with CGP to define context-generic providers that are generic over multiple *abstract* types.

Building Authentication Components

Suppose we want to build a simple authentication system using *bearer tokens* with an expiration time. To achieve this, we need to fetch the expiration time of a valid token and ensure that it is not in the past. A naive approach to implementing the authentication might look like the following:

```
pub mod traits {
    use anyhow::Error;
    use cgp::prelude::*;

    #[cgp_component(AuthTokenValidator)]
    pub trait CanValidateAuthToken {
        fn validate_auth_token(&self, auth_token: &str) -> Result<(), Error>;
    }

    #[cgp_component(AuthTokenExpiryFetcher)]
    pub trait CanFetchAuthTokenExpiry {
        fn fetch_auth_token_expiry(&self, auth_token: &str) -> Result<u64,
Error>;
    }

    #[cgp_component(CurrentTimeGetter)]
    pub trait HasCurrentTime {
        fn current_time(&self) -> Result<u64, Error>;
    }
}

pub mod impls {
    use std::time::{SystemTime, UNIX_EPOCH};

    use anyhow::{anyhow, Error};
    use cgp::prelude::*;

    use super::traits::*;

    #[cgp_new_provider]
    impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
    where
        Context: HasCurrentTime + CanFetchAuthTokenExpiry,
    {
        fn validate_auth_token(context: &Context, auth_token: &str) ->
Result<(), Error> {
            let now = context.current_time()?;

            let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

            if token_expiry < now {
                Ok(())
            } else {
                Err(anyhow!("auth token has expired"))
            }
        }
    }

    #[cgp_new_provider]
    impl<Context> CurrentTimeGetter<Context> for GetSystemTimestamp {
        fn current_time(_context: &Context) -> Result<u64, Error> {
            let now = SystemTime::now()
                .duration_since(UNIX_EPOCH)?
                .as_millis()
                .try_into()?;
```

```
            Ok(now)
        }
    }
}

pub mod contexts {
    use std::collections::BTreeMap;

    use anyhow::anyhow;
    use cgp::prelude::*;

    use super::impls::*;
    use super::traits::*;

    #[cgp_context]
    pub struct MockApp {
        pub auth_tokens_store: BTreeMap<String, u64>,
    }

    delegate_and_check_components! {
        CanUseMockApp for MockApp;
        MockAppComponents {
            CurrentTimeGetterComponent: GetSystemTimestamp,
            AuthTokenValidatorComponent: ValidateTokenIsNotExpired,
        }
    }

    #[cgp_provider]
    impl AuthTokenExpiryFetcher<MockApp> for MockAppComponents {
        fn fetch_auth_token_expiry(
            context: &MockApp,
            auth_token: &str,
        ) -> Result<u64, anyhow::Error> {
            context
                .auth_tokens_store
                .get(auth_token)
                .cloned()
                .ok_or_else(|| anyhow!("invalid auth token"))
        }
    }
}
```

In this example, we first define the `CanValidateAuthToken` trait, which serves as the primary API for validating authentication tokens. To facilitate the implementation of the validator, we also define the `CanFetchAuthTokenExpiry` trait, which is responsible for fetching the expiration time of an authentication token — assuming the token is valid. Finally, the `HasCurrentTime` trait is introduced to retrieve the current time.

Next, we define a context-generic provider, `ValidateTokenIsNotExpired`, which validates authentication tokens by comparing their expiration time with the current time. The provider fetches both the token's expiration time and the current time, and ensure that the token is still valid. Additionally, we define another context-generic provider, `GetSystemTimestamp`, which retrieves the current time using

`std::time::SystemTime::now()`.

For this demonstration, we introduce a concrete context, `MockApp`, which includes an `auth_tokens_store` field. This store is a mocked collection of authentication tokens with their respective expiration times, stored in a `BTreeMap`. We also implement the `AuthTokenExpiryFetcher` trait specifically for the `MockApp` context, which retrieves expiration times from the mocked `auth_tokens_store`. Lastly, we define the `CanUseMockApp` trait, ensuring that `MockApp` properly implements the `CanValidateAuthToken` trait through the provided wiring.

Abstract Types

The previous example demonstrates basic CGP techniques for implementing a reusable provider, `ValidateTokenIsNotExpired`, which can work with different concrete contexts. However, the method signatures are tied to specific types. For instance, we use `String` to represent the authentication token and `u64` to represent the Unix timestamp in milliseconds.

Common practice suggests that we should use distinct types to differentiate values from different domains, reducing the chance of mixing them up. A common approach in Rust is to use the *newtype pattern* to define wrapper types, like so:

```
pub struct AuthToken {
    value: String,
}

pub struct Time {
    value: u64,
}
```

While the newtype pattern helps abstract over underlying values, it doesn't fully generalize the code to work with different types. For example, instead of defining our own `Time` type with Unix timestamp semantics, we may want to use a datetime library such as `datetime` or `chrono`. The choice of library could depend on the specific use case of a concrete application.

A more flexible approach is to define an *abstract* `Time` type that allows us to implement context-generic providers compatible with *any* `Time` type chosen by the concrete context. This can be achieved in CGP by defining *type traits* that contain *associated types*:

```
use cgp::prelude::*;

#[cgp_component(TimeTypeProviderComponent)]
pub trait HasTimeType {
    type Time: Eq + Ord;
}

#[cgp_component(AuthTokenTypeProviderComponent)]
pub trait HasAuthTokenType {
    type AuthToken;
}
```

Here, we define the `HasTimeType` trait with an associated type `Time`, which is constrained to types that implement `Eq` and `Ord` so that they can be compared. Similarly, the `HasAuthTokenType` trait defines an associated type `AuthToken`, without any additional constraints.

Similar to regular trait methods, CGP allows us to auto-derive blanket implementations that delegate the associated types to providers using `HasCgpProvider` and `DelegateComponent`. Therefore, we can use `#[cgp_component]` on traits containing associated types as well.

With these type traits in place, we can now update our authentication components to leverage abstract types within the trait methods:

```
#[cgp_component(AuthTokenValidator)]
pub trait CanValidateAuthToken: HasAuthTokenType {
    fn validate_auth_token(&self, auth_token: &Self::AuthToken) -> Result<(),
Error>;
}

#[cgp_component(AuthTokenExpiryFetcher)]
pub trait CanFetchAuthTokenExpiry: HasAuthTokenType + HasTimeType {
    fn fetch_auth_token_expiry(&self, auth_token: &Self::AuthToken) ->
Result<Self::Time, Error>;
}

#[cgp_component(CurrentTimeGetter)]
pub trait HasCurrentTime: HasTimeType {
    fn current_time(&self) -> Result<Self::Time, Error>;
}
```

Here, we modify the `CanValidateAuthToken` trait to include `HasAuthTokenType` as a supertrait, allowing it to accept the abstract type `Self::AuthToken` as a method parameter. Likewise, `CanFetchAuthTokenExpiry` requires both `HasAuthTokenType` and `HasTimeType`, while `HasCurrentTime` only requires `HasTimeType`.

With the abstract types defined, we can now update `ValidateTokenIsNotExpired` to work generically with any `Time` and `AuthToken` types:

```rust
#[cgp_new_provider]
impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
where
    Context: HasCurrentTime + CanFetchAuthTokenExpiry,
{
    fn validate_auth_token(
        context: &Context,
        auth_token: &Context::AuthToken,
    ) -> Result<(), Error> {
        let now = context.current_time()?;

        let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

        if token_expiry < now {
            Ok(())
        } else {
            Err(anyhow!("auth token has expired"))
        }
    }
}
```

This example shows how CGP enables us to define context-generic providers that are not just generic over the context itself, but also over its associated types. Unlike traditional generic programming, where all generic parameters are specified positionally, CGP allows us to parameterize abstract types using *names* via associated types.

Defining Abstract Type Traits with `#[cgp_type]`

The type traits `HasTimeType` and `HasAuthTokenType` share a similar structure, and as you define more abstract types, this boilerplate can become tedious. To streamline the process, the `cgp` crate provides the `#[cgp_type]` macro, which simplifies type trait definitions.

Here's how you can define the same types with `#[cgp_type]`:

```
use cgp::prelude::*;

#[cgp_type {
    provider: TimeTypeProvider,
}]
pub trait HasTimeType {
    type Time: Eq + Ord;
}

#[cgp_type {
    provider: AuthTokenTypeProvider,
}]
pub trait HasAuthTokenType {
    type AuthToken;
}
```

The `#[cgp_type]` macro works with a CGP trait that contains a single non-generic associated type. It is an extension over `#[cgp_component]`, and generate additional constructs that make it easy to work with abstract types in CGP. When no argument is given, `#[cgp_type]` would default to generate a provider with name `{Type}TypeProvider`, and a component name `{Type}TypeProviderComponent`, where `{Type}` is the name of the associated type in the trait. So the above example can be shortened to:

```
use cgp::prelude::*;

#[cgp_type]
pub trait HasTimeType {
    type Time: Eq + Ord;
}

#[cgp_type]
pub trait HasAuthTokenType {
    type AuthToken;
}
```

We will explore in a moment how using `#[cgp_type]` with a single associated type bring more convenience, as compared to alternative approaches.

Impl-Side Associated Type Constraints

The dependency-injection capabilities of CGP opens up new choices of how to design the abstract type interfaces. Consider the earlier definition of `HasTimeType`:

```
#[cgp_type]
pub trait HasTimeType {
    type Time: Eq + Ord;
}
```

Here, the associated `Time` type is constrained by `Eq + Ord`. This means that all concrete implementations of `Time` must satisfy these constraints, regardless of whether they are actually required by the providers. In fact, if we revisit our previous examples, we notice that the `Eq` constraint isn't used anywhere.

Such overly restrictive constraints can become a bottleneck as the application evolves. As complexity increases, it's common to require additional traits on `Time`, such as `Debug + Display + Clone + Hash + Serialize + Deserialize` and so on. Imposing these constraints globally limits flexibility and makes it harder to adapt to changing requirements.

Fortunately, CGP allows us to apply the same principle of impl-side dependencies to associated type constraints. Consider the following example:

```rust
#[cgp_type]
pub trait HasTimeType {
    type Time: Eq + Ord;
}

#[cgp_new_provider]
impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
where
    Context: HasCurrentTime + CanFetchAuthTokenExpiry,
    Context::Time: Ord,
{
    fn validate_auth_token(
        context: &Context,
        auth_token: &Context::AuthToken,
    ) -> Result<(), Error> {
        let now = context.current_time()?;

        let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

        if token_expiry < now {
            Ok(())
        } else {
            Err(anyhow!("auth token has expired"))
        }
    }
}
```

In this example, we redefine `HasTimeType::Time` *without* any constraints. Instead, we specify the constraint `Context::Time: Ord` in the provider implementation for `ValidateTokenIsNotExpired`. This ensures that the `ValidateTokenIsNotExpired` provider can compare the token expiry time using `Ord`, while avoiding unnecessary global constraints on `Time`.

By applying constraints on the implementation side, we can conditionally require `HasTimeType::Time` to implement `Ord`, but only when the `ValidateTokenIsNotExpired` provider is in use. This approach allows abstract types to scale flexibly alongside generic

context types, enabling the same CGP patterns to be applied to abstract types.

In some cases, it can still be convenient to include constraints (e.g., `Debug`) directly on an associated type, especially if those constraints are nearly universal across providers. Additionally, current Rust error reporting often produces clearer error messages when constraints are defined at the associated type level, as opposed to being deferred to the implementation.

Ultimately, CGP does not prevent its users from preferring one design approach over another. The minimalistic abstract type design is one that you will likely see often in CGP code, particularly in this book. However, do not hesitate to include addititional trait bounds based on your requirements and preferences!

Type Providers

With type abstraction in place, we can define context-generic providers for the `Time` and `AuthToken` abstract types. For example, we can create a provider that uses `std::time::Instant` as the `Time` type:

```
pub struct UseInstant;

#[cgp_provider]
impl<Context> TimeTypeProvider<Context> for UseInstant {
    type Time = Instant;
}

#[cgp_provider]
impl<Context> CurrentTimeGetter<Context> for UseInstant
where
    Context: HasTimeType<Time = Instant>,
{
    fn current_time(_context: &Context) -> Result<Instant, Error> {
        Ok(Instant::now())
    }
}
```

Here, the `UseInstant` provider implements `TimeTypeProvider` for any `Context` type by setting the associated type `Time` to `Instant`. Additionally, it implements `CurrentTimeGetter` for any `Context`, *provided* that `Context::Time` is `Instant`. This type equality constraint works similarly to regular implementation-side dependencies and is frequently used for scope-limited access to a concrete type associated with an abstract type.

The type equality constraint is necessary because a given context might not always use `UseInstant` as the provider for `TimeTypeProvider`. Instead, the context could choose a different provider that uses another type to represent `Time`. Consequently, `UseInstant`

can only implement `CurrentTimeGetter` if the `Context` uses it or another provider that also uses `Instant` as its `Time` type.

Aside from `Instant`, we can also define alternative providers for `Time`, using other types like `datetime::LocalDateTime`:

```
pub struct UseLocalDateTime;

#[cgp_provider]
impl<Context> TimeTypeProvider<Context> for UseLocalDateTime {
    type Time = LocalDateTime;
}

#[cgp_provider]
impl<Context> CurrentTimeGetter<Context> for UseLocalDateTime
where
    Context: HasTimeType<Time = LocalDateTime>,
{
    fn current_time(_context: &Context) -> Result<LocalDateTime, Error> {
        Ok(LocalDateTime::now())
    }
}
```

Since our application only requires the `Time` type to implement `Ord` and the ability to retrieve the current time, we can easily swap between different time providers, as long as they meet these dependencies. As the application evolves, additional constraints might be introduced on the Time type, potentially limiting the available concrete time types. However, with CGP, we can incrementally introduce new dependencies based on the application's needs, avoiding premature restrictions caused by unused requirements.

Similarly, for the abstract `AuthToken` type, we can define a context-generic provider `AuthTokenTypeProvider` that uses `String` as its implementation:

```
#[cgp_new_provider]
impl<Context> AuthTokenTypeProvider<Context> for UseStringAuthToken {
    type AuthToken = String;
}
```

Comparison to Newtype Pattern

Let's talk about abstract types and how they compare to the well-known newtype pattern in Rust. Abstract types provide a compelling alternative.

A key difference is that abstract types often allow you to work directly with plain, convenient types like `String`, whereas the newtype pattern typically requires wrapping your value within a dedicated struct.

Within the CGP philosophy, we lean towards a different approach than the common advice to strictly newtype *every* domain concept. This is particularly true when you're writing highly context-generic code.

The reason is simple: abstract types, defined by traits that specify their required behavior and structure, effectively provide the same level of encapsulation and type safety that newtypes offer. They protect the underlying raw values through their defined interfaces, often making the extra layer of newtype wrapping unnecessary boilerplate.

The choice between abstract types, newtypes, or using both together isn't about a rigid "right" or "wrong." It's a flexible decision based on your specific project requirements, team conventions, and personal preference – choosing the best tool for the job at hand.

Just keep in mind that abstract types are a core and widely used pattern in the CGP ecosystem, and you'll see them frequently demonstrated throughout this book.

Comparison to Newtype Pattern

Abstract types serve as an alternative to the newtype pattern. Compared to the newtype pattern, we can use plain `String` values directly, without wrapping them in a newtype struct.

Contrary to common wisdom, in CGP, we place less emphasis on wrapping every domain type in a newtype. This is particularly true when most of the application is written in a context-generic style. The reason is simple: abstract types, when used within generic code, effectively provide the same level of encapsulation and type safety that newtypes offer. They protect the underlying raw values through their defined interfaces, often making the extra layer of newtype wrapping unnecessary boilerplate.

The choice between abstract types, newtypes, or using both together isn't about a rigid "right" or "wrong." It's a flexible decision based on your specific project requirements, team conventions, and personal preference – choosing the best tool for the job at hand.

Just keep in mind that abstract types are a core and widely used pattern in the CGP ecosystem, and you'll see them frequently demonstrated throughout this book.

The `UseType` Pattern

Implementing type providers can quickly become repetitive as the number of abstract types grows. For example, to use `String` as the `AuthToken` type, we first need to define a new struct, `UseStringAuthToken`, and then implement `AuthTokenTypeProvider` for it. To streamline this process, the `cgp_type!` macro simplifies the implementation by

automatically generating a provider using the *UseType* pattern. The generated implementation looks like this:

```
pub struct UseType<Type>(pub PhantomData<Type>);

#[cgp_provider]
impl<Context, AuthToken> AuthTokenTypeProvider<Context> for
UseType<AuthToken> {
    type AuthToken = AuthToken;
}
```

Here, `UseType` is a *marker* type with a generic parameter `Type`, representing the type to be used for a given type trait. Since `PhantomData` is its only field, `UseType` is never intended to be used as a runtime value. The generic implementation of `AuthTokenTypeProvider` for `UseType` ensures that the AuthToken type is directly set to the `Type` parameter of `UseType`.

With this generic implementation, we can redefine `UseStringAuthToken` as a simple type alias for `UseType<String>`:

```
type UseStringAuthToken = UseType<String>;
```

In fact, we can even skip defining type aliases altogether and use `UseType` directly in the `delegate_components` macro when wiring type providers.

The `UseType` struct is included in the `cgp` crate, and when you define an abstract type using the `cgp_type!` macro, the corresponding generic `UseType` implementation is automatically derived. This makes `UseType` a powerful tool for simplifying component wiring and reducing boilerplate in your code.

Putting It Altogether

With all the pieces in place, we can now apply what we've learned and refactor our naive authentication components to utilize abstract types, as shown below:

```
pub mod traits {
    use anyhow::Error;
    use cgp::prelude::*;

    #[cgp_type]
    pub trait HasTimeType {
        type Time: Eq + Ord;
    }

    #[cgp_type]
    pub trait HasAuthTokenType {
        type AuthToken;
    }

    #[cgp_component(AuthTokenValidator)]
    pub trait CanValidateAuthToken: HasAuthTokenType {
        fn validate_auth_token(&self, auth_token: &Self::AuthToken) ->
Result<(), Error>;
    }

    #[cgp_component(AuthTokenExpiryFetcher)]
    pub trait CanFetchAuthTokenExpiry: HasAuthTokenType + HasTimeType {
        fn fetch_auth_token_expiry(
            &self,
            auth_token: &Self::AuthToken,
        ) -> Result<Self::Time, Error>;
    }

    #[cgp_component(CurrentTimeGetter)]
    pub trait HasCurrentTime: HasTimeType {
        fn current_time(&self) -> Result<Self::Time, Error>;
    }
}

pub mod impls {
    use anyhow::{anyhow, Error};
    use cgp::prelude::*;
    use datetime::LocalDateTime;

    use super::traits::*;

    #[cgp_new_provider]
    impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
    where
        Context: HasCurrentTime + CanFetchAuthTokenExpiry,
        Context::Time: Ord,
    {
        fn validate_auth_token(
            context: &Context,
            auth_token: &Context::AuthToken,
        ) -> Result<(), Error> {
            let now = context.current_time()?;

            let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

            if token_expiry < now {
```

```
                Ok(())
            } else {
                Err(anyhow!("auth token has expired"))
            }
        }
    }

    pub struct UseLocalDateTime;

    #[cgp_provider]
    impl<Context> TimeTypeProvider<Context> for UseLocalDateTime {
        type Time = LocalDateTime;
    }

    #[cgp_provider]
    impl<Context> CurrentTimeGetter<Context> for UseLocalDateTime
    where
        Context: HasTimeType<Time = LocalDateTime>,
    {
        fn current_time(_context: &Context) -> Result<LocalDateTime, Error> {
            Ok(LocalDateTime::now())
        }
    }
}

pub mod contexts {
    use std::collections::BTreeMap;

    use anyhow::anyhow;
    use cgp::prelude::*;
    use datetime::LocalDateTime;

    use super::impls::*;
    use super::traits::*;

    #[cgp_context]
    pub struct MockApp {
        pub auth_tokens_store: BTreeMap<String, LocalDateTime>,
    }

    delegate_and_check_components! {
        CanUseMockApp for MockApp;
        MockAppComponents {
            [
                TimeTypeProviderComponent,
                CurrentTimeGetterComponent,
            ]: UseLocalDateTime,
            AuthTokenTypeProviderComponent: UseType<String>,
            AuthTokenValidatorComponent: ValidateTokenIsNotExpired,
        }
    }

    #[cgp_provider]
    impl AuthTokenExpiryFetcher<MockApp> for MockAppComponents {
        fn fetch_auth_token_expiry(
            context: &MockApp,
            auth_token: &String,
```

```
        ) -> Result<LocalDateTime, anyhow::Error> {
            context
                .auth_tokens_store
                .get(auth_token)
                .cloned()
                .ok_or_else(|| anyhow!("invalid auth token"))
        }
    }
}
```

Compared to our earlier approach, it is now much easier to update the `MockApp` context to use different time and auth token providers. If different use cases require distinct concrete types, we can easily define additional context types with different configurations, all without duplicating the core logic.

So far, we have applied abstract types to the `Time` and `AuthToken` types, but we are still relying on the concrete `anyhow::Error` type. In the next chapter, we will explore error handling in depth and learn how to use abstract error types to improve the way application errors are managed.

Error Handling

Rust introduces a modern approach to error handling through the use of the `Result` type, which explicitly represents errors. Unlike implicit exceptions commonly used in other mainstream languages, the `Result` type offers several advantages. It clearly indicates when errors may occur and specifies the type of errors that might be encountered when calling a function. However, the Rust community has yet to reach a consensus on the ideal *error type* to use within a `Result`.

Choosing an appropriate error type is challenging because different applications have distinct requirements. For instance, should the error include stack traces? Can it be compatible with no_std environments? How should the error message be presented? Should it include *structured metadata* for introspection or specialized logging? How can different errors be distinguished to determine whether an operation should be retried? How can error sources from various libraries be composed or *flattened* effectively? These and other concerns complicate the decision-making process.

Because of these cross-cutting concerns, discussions in the Rust community about finding a universally optimal error type are never ending. Currently, the ecosystem tends to favor libraries like `anyhow` that store error values using some form of *dynamic typing*. While convenient, these approaches sacrifice some benefits of static typing, such as the ability to determine at compile time whether a function cannot produce certain errors.

CGP offers an alternative approach to error handling: using *abstract* error types within `Result` alongside a context-generic mechanism for *raising errors* without requiring a specific error type. In this chapter, we will explore this new approach, demonstrating how it allows error handling to be tailored to an application's precise needs.

Abstract Error Type

In the previous chapter, we explored how to use associated types with CGP to define abstract types. Similarly to abstract types like `Time` and `AuthToken`, we can define an abstract `Error` type as follows:

```
use core::fmt::Debug;

use cgp::prelude::*;

#[cgp_type]
pub trait HasErrorType {
    type Error: Debug;
}
```

The `HasErrorType` trait is particularly significant because it serves as a standard type API for *all* CGP components that involve abstract errors. Its definition is intentionally minimal, consisting of a single associated type, `Error`, constrained by `Debug` by default. This `Debug` constraint was chosen because many Rust APIs, such as `Result::unwrap`, rely on error types implementing `Debug`.

Given its ubiquity, the `HasErrorType` trait is included as part of the `cgp` crate and is available in the prelude. Therefore, we will use the version provided by `cgp` rather than redefining it locally in subsequent examples.

Building on the example from the previous chapter, we can update authentication components to leverage the abstract error type defined by `HasErrorType`:

```
#[cgp_component {
    provider: AuthTokenValidator,
}]
pub trait CanValidateAuthToken: HasAuthTokenType + HasErrorType {
    fn validate_auth_token(&self, auth_token: &Self::AuthToken) -> Result<(),
Self::Error>;
}

#[cgp_component {
    provider: AuthTokenExpiryFetcher,
}]
pub trait CanFetchAuthTokenExpiry: HasAuthTokenType + HasTimeType +
HasErrorType {
    fn fetch_auth_token_expiry(
        &self,
        auth_token: &Self::AuthToken,
    ) -> Result<Self::Time, Self::Error>;
}

#[cgp_component {
    provider: CurrentTimeGetter,
}]
pub trait HasCurrentTime: HasTimeType + HasErrorType {
    fn current_time(&self) -> Result<Self::Time, Self::Error>;
}
```

In these examples, each trait now includes `HasErrorType` as a supertrait, and methods return `Self::Error` in the `Result` type instead of relying on a concrete type like `anyhow::Error`. This abstraction allows greater flexibility and customization, enabling components to adapt their error handling to the specific needs of different contexts.

Raising Errors With `From`

After adopting abstract errors in our component interfaces, the next challenge is handling

these abstract errors in context-generic providers. With CGP, this is achieved by leveraging impl-side dependencies and adding constraints to the `Error` type, such as requiring it to implement `From`. This allows for the conversion of a source error into an abstract error value.

For example, we can modify the `ValidateTokenIsNotExpired` provider to convert a source error, `&'static str`, into `Context::Error` when an authentication token has expired:

```
#[cgp_new_provider]
impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
where
    Context: HasCurrentTime + CanFetchAuthTokenExpiry + HasErrorType,
    Context::Time: Ord,
    Context::Error: From<&'static str>
{
    fn validate_auth_token(
        context: &Context,
        auth_token: &Context::AuthToken,
    ) -> Result<(), Context::Error> {
        let now = context.current_time()?;

        let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

        if token_expiry < now {
            Ok(())
        } else {
            Err("auth token has expired".into())
        }
    }
}
```

This example demonstrates how CGP simplifies "stringy" error handling in context-generic providers by delegating the conversion from strings to concrete error values to the application. While using string errors is generally not a best practice, it is useful during the prototyping phase when precise error handling strategies are not yet established.

CGP encourages an iterative approach to error handling. Developers can begin with string errors for rapid prototyping and transition to structured error handling as the application matures. For example, we can replace the string error with a custom error type like `ErrAuthTokenHasExpired`:

```rust
#[cgp_new_provider]
impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
where
    Context: HasCurrentTime + CanFetchAuthTokenExpiry + HasErrorType,
    Context::Time: Ord,
    Context::Error: From<ErrAuthTokenHasExpired>
{
    fn validate_auth_token(
        context: &Context,
        auth_token: &Context::AuthToken,
    ) -> Result<(), Context::Error> {
        let now = context.current_time()?;

        let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

        if token_expiry < now {
            Ok(())
        } else {
            Err(ErrAuthTokenHasExpired.into())
        }
    }
}

#[derive(Debug)]
pub struct ErrAuthTokenHasExpired;

impl Display for ErrAuthTokenHasExpired {
    fn fmt(&self, f: &mut core::fmt::Formatter<'_>) -> core::fmt::Result {
        write!(f, "auth token has expired")
    }
}
```

In this example, we introduced the `ErrAuthTokenHasExpired` type to represent the specific error of an expired authentication token. The `AuthTokenValidator` implementation requires `Context::Error` to implement `From<ErrAuthTokenHasExpired>` for conversion to the abstract error type. Additionally, `ErrAuthTokenHasExpired` implements both `Debug` and `Display`, allowing applications to present and log the error meaningfully.

CGP facilitates defining provider-specific error types like `ErrAuthTokenHasExpired` without burdening the provider with embedding these errors into the application's overall error handling strategy. With impl-side dependencies, constraints like `Context::Error: From<ErrAuthTokenHasExpired>` apply only when the application uses a specific provider. If an application employs a different provider to implement `AuthTokenValidator`, it does not need to handle the `ErrAuthTokenHasExpired` error.

Raising Errors using `CanRaiseError`

In the previous section, we used the `From` constraint in the `ValidateTokenIsNotExpired` provider to raise errors such as `&'static str` or `ErrAuthTokenHasExpired`. While this approach is elegant, we quickly realize it doesn't work with common error types like `anyhow::Error`. This is because `anyhow::Error` only provides a blanket From implementation only for types that implement `core::error::Error + Send + Sync + 'static`.

This restriction is a common pain point when using error libraries like `anyhow`. The reason for this limitation is that without CGP, a type like `anyhow::Error` cannot provide multiple blanket `From` implementations without causing conflicts. As a result, using `From` can leak abstractions, forcing custom error types like `ErrAuthTokenHasExpired` to implement common traits like `core::error::Error`. Another challenge is that ownership rules prevent supporting custom `From` implementations for non-owned types like `String` and `&str`.

To address these issues, we recommend using a more flexible — though slightly more verbose—approach with CGP: the `CanRaiseError` trait, rather than relying on `From` for error conversion. Here's how we define it:

```
#[cgp_component(ErrorRaiser)]
pub trait CanRaiseError<SourceError>: HasErrorType {
    fn raise_error(e: SourceError) -> Self::Error;
}
```

The `CanRaiseError` trait has a *generic parameter* `SourceError`, representing the source error type that will be converted into the abstract error type `HasErrorType::Error`. By making it a generic parameter, this allows a context to raise multiple source error types and convert them into the abstract error.

Since raising errors is common in most CGP code, the `CanRaiseError` trait is included in the CGP prelude, so we don’t need to define it manually.

We can now update the `ValidateTokenIsNotExpired` provider to use `CanRaiseError` instead of `From` for error handling, raising a source error like `&'static str`:

```rust
#[cgp_new_provider]
impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
where
    Context: HasCurrentTime + CanFetchAuthTokenExpiry +
CanRaiseError<&'static str>,
    Context::Time: Ord,
{
    fn validate_auth_token(
        context: &Context,
        auth_token: &Context::AuthToken,
    ) -> Result<(), Context::Error> {
        let now = context.current_time()?;

        let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

        if token_expiry < now {
            Ok(())
        } else {
            Err(Context::raise_error("auth token has expired"))
        }
    }
}
```

In this updated implementation, we replace the `Context: HasErrorType` constraint with `Context: CanRaiseError<&'static str>`. Since `HasErrorType` is a supertrait of `CanRaiseError`, we only need to include `CanRaiseError` in the constraint to automatically include `HasErrorType`. We also use the `Context::raise_error` method to convert the string `"auth token has expired"` into `Context::Error`.

This approach avoids the limitations of `From` and offers greater flexibility for error handling in CGP, especially when working with third-party error types like `anyhow::Error`.

Context-Generic Error Raisers

By defining the `CanRaiseError` trait using CGP, we overcome the limitations of `From` and enable context-generic error raisers that work across various source error types. For instance, we can create a context-generic error raiser for `anyhow::Error` as follows:

```rust
use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent, HasErrorType};

#[cgp_new_provider]
impl<Context, SourceError> ErrorRaiser<Context, SourceError> for
RaiseAnyhowError
where
    Context: HasErrorType<Error = anyhow::Error>,
    SourceError: core::error::Error + Send + Sync + 'static,
{
    fn raise_error(e: SourceError) -> anyhow::Error {
        e.into()
    }
}
```

Here, `RaiseAnyhowError` is a provider that implements the `ErrorRaiser` trait with generic `Context` and `SourceError`. The implementation is valid only if the `Context` implements `HasErrorType` and implements `Context::Error` as `anyhow::Error`. Additionally, the `SourceError` must satisfy `core::error::Error + Send + Sync + 'static`, which is necessary for the `From` implementation provided by `anyhow::Error`. Inside the method body, the source error is converted into `anyhow::Error` using `e.into()` since the required constraints are already satisfied.

For a more generalized approach, we can create a provider that works with *any* error type supporting `From`:

```rust
use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent, HasErrorType};

#[cgp_new_provider]
impl<Context, SourceError> ErrorRaiser<Context, SourceError> for RaiseFrom
where
    Context: HasErrorType,
    Context::Error: From<SourceError>,
{
    fn raise_error(e: SourceError) -> Context::Error {
        e.into()
    }
}
```

This implementation requires the `Context` to implement `HasErrorType` and the `Context::Error` type to implement `From<SourceError>`. With these constraints in place, this provider allows errors to be raised from any source type to `Context::Error` using `From`, without requiring explicit coupling in providers like `ValidateTokenIsNotExpired`.

The introduction of `CanRaiseError` might seem redundant when it ultimately relies on `From` in some cases. However, the purpose of this indirection is to enable *alternative* mechanisms for converting errors when `From` is insufficient or unavailable. For example, we can define an error raiser for `anyhow::Error` that uses the `Debug` trait instead of `From`:

```rust
use core::fmt::Debug;

use anyhow::anyhow;
use cgp::prelude::*;
use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent, HasErrorType};

#[cgp_new_provider]
impl<Context, SourceError> ErrorRaiser<Context, SourceError> for
DebugAnyhowError
where
    Context: HasErrorType<Error = anyhow::Error>,
    SourceError: Debug,
{
    fn raise_error(e: SourceError) -> anyhow::Error {
        anyhow!("{e:?}")
    }
}
```

In this implementation, the `DebugAnyhowError` provider raises any source error into an `anyhow::Error`, as long as the source error implements `Debug`. The `raise_error` method uses the `anyhow!` macro and formats the source error using the `Debug` trait. This approach allows a concrete context to use providers like `ValidateTokenIsNotExpired` while relying on `DebugAnyhowError` to raise source errors such as `&'static str` or `ErrAuthTokenHasExpired`, which only implement `Debug` or `Display`.

The `cgp-error-anyhow` Crate

The CGP project provides the `cgp-error-anyhow` crate, which includes the anyhow-specific providers discussed in this chapter. These constructs are offered as a separate crate rather than being part of the core `cgp` crate to avoid adding `anyhow` as a mandatory dependency.

In addition, CGP offers other error crates tailored to different error handling libraries. The `cgp-error-eyre` crate supports `eyre::Error`, while the `cgp-error-std` crate works with `Box<dyn core::error::Error>`.

As demonstrated in this chapter, CGP allows projects to easily switch between error handling implementations without being tightly coupled to a specific error type. For instance, if the application needs to run in a resource-constrained environment, replacing `cgp-error-anyhow` with `cgp-error-std` in the component wiring enables the application to use the simpler `Box<dyn Error>` type for error handling.

Putting It Altogether

With the use of `HasErrorType`, `CanRaiseError`, and `cgp-error-anyhow`, we can now refactor the full example from the previous chapter, and make it generic over the error type:

```
pub mod traits {
    use cgp::prelude::*;

    #[cgp_type]
    pub trait HasTimeType {
        type Time: Eq + Ord;
    }

    #[cgp_type]
    pub trait HasAuthTokenType {
        type AuthToken;
    }

    #[cgp_component(AuthTokenValidator)]
    pub trait CanValidateAuthToken: HasAuthTokenType + HasErrorType {
        fn validate_auth_token(&self, auth_token: &Self::AuthToken) ->
Result<(), Self::Error>;
    }

    #[cgp_component(AuthTokenExpiryFetcher)]
    pub trait CanFetchAuthTokenExpiry: HasAuthTokenType + HasTimeType +
HasErrorType {
        fn fetch_auth_token_expiry(
            &self,
            auth_token: &Self::AuthToken,
        ) -> Result<Self::Time, Self::Error>;
    }

    #[cgp_component(CurrentTimeGetter)]
    pub trait HasCurrentTime: HasTimeType + HasErrorType {
        fn current_time(&self) -> Result<Self::Time, Self::Error>;
    }
}

pub mod impls {
    use core::fmt::Debug;

    use anyhow::anyhow;
    use cgp::prelude::*;
    use datetime::LocalDateTime;

    use super::traits::*;

    #[derive(Debug)]
    pub struct ErrAuthTokenHasExpired;

    #[cgp_new_provider]
    impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
    where
        Context: HasCurrentTime + CanFetchAuthTokenExpiry +
CanRaiseError<ErrAuthTokenHasExpired>,
        Context::Time: Ord,
    {
        fn validate_auth_token(
            context: &Context,
            auth_token: &Context::AuthToken,
```

```
        ) -> Result<(), Context::Error> {
            let now = context.current_time()?;

            let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

            if token_expiry < now {
                Ok(())
            } else {
                Err(Context::raise_error(ErrAuthTokenHasExpired))
            }
        }
    }

    pub struct UseLocalDateTime;

    #[cgp_provider]
    impl<Context> TimeTypeProvider<Context> for UseLocalDateTime {
        type Time = LocalDateTime;
    }

    #[cgp_provider]
    impl<Context> CurrentTimeGetter<Context> for UseLocalDateTime
    where
        Context: HasTimeType<Time = LocalDateTime> + HasErrorType,
    {
        fn current_time(_context: &Context) -> Result<LocalDateTime,
Context::Error> {
            Ok(LocalDateTime::now())
        }
    }
}

pub mod contexts {
    use std::collections::BTreeMap;

    use anyhow::anyhow;
    use cgp::core::error::{ErrorRaiserComponent, ErrorTypeProviderComponent};
    use cgp::prelude::*;
    use cgp_error_anyhow::{UseAnyhowError, DebugAnyhowError};
    use datetime::LocalDateTime;

    use super::impls::*;
    use super::traits::*;

    #[cgp_context]
    pub struct MockApp {
        pub auth_tokens_store: BTreeMap<String, LocalDateTime>,
    }

    delegate_and_check_components! {
        CanUseMockApp for MockApp;
        MockAppComponents {
            ErrorTypeProviderComponent:
                UseAnyhowError,
            ErrorRaiserComponent:
                DebugAnyhowError,
            [
```

```
                TimeTypeProviderComponent,
                CurrentTimeGetterComponent,
            ]: UseLocalDateTime,
            AuthTokenTypeProviderComponent: UseType<String>,
            AuthTokenValidatorComponent: ValidateTokenIsNotExpired,
        }
    }

    #[cgp_provider]
    impl AuthTokenExpiryFetcher<MockApp> for MockAppComponents {
        fn fetch_auth_token_expiry(
            context: &MockApp,
            auth_token: &String,
        ) -> Result<LocalDateTime, anyhow::Error> {
            context
                .auth_tokens_store
                .get(auth_token)
                .cloned()
                .ok_or_else(|| anyhow!("invalid auth token"))
        }
    }
}
```

In the updated code, we refactored `ValidateTokenIsNotExpired` to use `CanRaiseError<ErrAuthTokenHasExpired>`, with `ErrAuthTokenHasExpired` implementing only `Debug`. Additionally, we use the provider `UseAnyhowError` from `cgp-error-anyhow`, which implements `ProvideErrorType` by setting `Error` to `anyhow::Error`.

In the component wiring for `MockAppComponents`, we wire up `ErrorTypeProviderComponent` with `UseAnyhowError` and `ErrorRaiserComponent` with `DebugAnyhowError`. In the context-specific implementation `AuthTokenExpiryFetcher<MockApp>`, we can now use `anyhow::Error` directly, since Rust already knows that `MockApp::Error` is the same type as `anyhow::Error`.

Conclusion

In this chapter, we provided a high-level overview of how error handling in CGP differs significantly from traditional error handling done in Rust. By utilizing abstract error types with `HasErrorType`, we can create providers that are generic over the concrete error type used by an application. The `CanRaiseError` trait allows us to implement context-generic error raisers, overcoming the limitations of non-overlapping implementations and enabling us to work with source errors that only implement traits like `Debug`.

However, error handling is a complex subject, and CGP abstractions such as `HasErrorType` and `CanRaiseError` are just the foundation for addressing this complexity. There are additional details related to error handling that we will explore in the upcoming chapters, preparing us to handle errors effectively in real-world

applications.

Delegated Error Raisers

In the previous chapter, we defined context-generic error raisers such as `RaiseFrom` and `DebugAnyhowError`, which can be used to raise any source error that satisfies certain constraints. However, in the main wiring for `MockAppComponents`, we could only select a specific provider for the `ErrorRaiserComponent`.

In more complex applications, we might want to handle different source errors in different ways, depending on the type of the source error. For example, we might use `RaiseFrom` when a `From` instance is available, and default to `DebugAnyhowError` for cases where the source error implements `Debug`.

In this chapter, we will introduce the *`UseDelegate`* pattern, which provides a declarative approach to handle errors differently based on the source error type.

Ad Hoc Error Raisers

One way to handle source errors differently is by defining an error raiser provider with explicit implementations for each source error. For example:

```rust
use core::convert::Infallible;
use core::num::ParseIntError;

use anyhow::anyhow;
use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent};
use cgp::prelude::*;

pub struct MyErrorRaiser;

#[cgp_provider]
impl<Context> ErrorRaiser<Context, anyhow::Error> for MyErrorRaiser
where
    Context: HasErrorType<Error = anyhow::Error>,
{
    fn raise_error(e: anyhow::Error) -> anyhow::Error {
        e
    }
}

#[cgp_provider]
impl<Context> ErrorRaiser<Context, Infallible> for MyErrorRaiser
where
    Context: HasErrorType,
{
    fn raise_error(e: Infallible) -> Context::Error {
        match e {}
    }
}

#[cgp_provider]
impl<Context> ErrorRaiser<Context, std::io::Error> for MyErrorRaiser
where
    Context: HasErrorType<Error = anyhow::Error>,
{
    fn raise_error(e: std::io::Error) -> anyhow::Error {
        e.into()
    }
}

#[cgp_provider]
impl<Context> ErrorRaiser<Context, ParseIntError> for MyErrorRaiser
where
    Context: HasErrorType<Error = anyhow::Error>,
{
    fn raise_error(e: ParseIntError) -> anyhow::Error {
        e.into()
    }
}

#[cgp_provider]
impl<Context> ErrorRaiser<Context, ErrAuthTokenHasExpired> for MyErrorRaiser
where
    Context: HasErrorType<Error = anyhow::Error>,
{
    fn raise_error(e: ErrAuthTokenHasExpired) -> anyhow::Error {
```

```
            anyhow!("{e:?}")
        }
    }

    #[cgp_provider]
    impl<Context> ErrorRaiser<Context, String> for MyErrorRaiser
    where
        Context: HasErrorType<Error = anyhow::Error>,
    {
        fn raise_error(e: String) -> anyhow::Error {
            anyhow!("{e}")
        }
    }

    #[cgp_provider]
    impl<'a, Context> ErrorRaiser<Context, &'a str> for MyErrorRaiser
    where
        Context: HasErrorType<Error = anyhow::Error>,
    {
        fn raise_error(e: &'a str) -> anyhow::Error {
            anyhow!("{e}")
        }
    }
```

In this example, we define the provider `MyErrorRaiser` with explicit `ErrorRaiser` implementations for a set of source error types, assuming that the abstract `Context::Error` is `anyhow::Error`.

With explicit implementations, `MyErrorRaiser` handles different source errors in various ways. When raising a source error of type `anyhow::Error`, we simply return `e` because `Context::Error` is also `anyhow::Error`. For `Infallible`, we handle the error by matching the empty case. For `std::io::Error` and `ParseIntError`, we rely on the `From` instance, as they satisfy the constraint `core::error::Error + Send + Sync + 'static`. When raising `ErrAuthTokenHasExpired`, we use the `anyhow!` macro to format the error with the `Debug` instance. For `String` and `&'a str`, we use `anyhow!` to format the error with the `Display` instance.

While defining explicit `ErrorRaiser` implementations provides a high degree of flexibility, it also requires a significant amount of repetitive boilerplate. Since we've already defined various generic error raisers, it would be beneficial to find a way to *delegate* error handling to different error raisers based on the source error type.

`UseDelegate` Pattern

When examining the patterns for implementing custom error raisers, we notice similarities to the provider delegation pattern we covered in an earlier chapter. In fact, with a bit of indirection, we can reuse `DelegateComponent` to delegate the handling of

source errors:

```
use core::marker::PhantomData;

use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent};
use cgp::prelude::*;

pub struct UseDelegate<Components>(pub PhantomData<Components>);

#[cgp_provider]
impl<Context, SourceError, Components> ErrorRaiser<Context, SourceError> for
UseDelegate<Components>
where
    Context: HasErrorType,
    Components: DelegateComponent<SourceError>,
    Components::Delegate: ErrorRaiser<Context, SourceError>,
{
    fn raise_error(e: SourceError) -> Context::Error {
        Components::Delegate::raise_error(e)
    }
}
```

Let's walk through the code step by step. First, we define the `UseDelegate` struct with a phantom `Components` parameter. `UseDelegate` serves as a *marker* type for implementing the trait-specific component delegation pattern. Here, we implement `ErrorRaiser` for `UseDelegate`, allowing it to act as a context-generic provider for `ErrorRaiser` under specific conditions.

Within the implementation, we specify that for any context `Context`, source error `SourceError`, and error raiser provider `Components`, `UseDelegate<Components>` implements `ErrorRaiser<Context, SourceError>` if `Components` implements `DelegateComponent<SourceError>`. Additionally, the delegate `Components::Delegate` must also implement `ErrorRaiser<Context, SourceError>`. Inside the `raise_error` method, we delegate the implementation to `Components::Delegate::raise_error`.

In simpler terms, `UseDelegate<Components>` implements `ErrorRaiser<Context, SourceError>` if there is a delegated provider `ErrorRaiser<Context, SourceError>` from `Components` via `SourceError`.

We can better understand this by looking at a concrete example. Using `UseDelegate`, we can declaratively dispatch errors as follows:

```
delegate_components! {
    new MyErrorRaiserComponents {
        [
            std::io::Error,
            ParseIntError,
        ]:
            RaiseFrom,
        [
            ErrAuthTokenHasExpired,
        ]:
            DebugAnyhowError,
    }
}

pub type MyErrorRaiser = UseDelegate<MyErrorRaiserComponents>;
```

In this example, we first define `MyErrorRaiserComponents` and use `delegate_components!` to map source error types to the error raiser providers we wish to use. Then, we redefine `MyErrorRaiser` to be `UseDelegate<MyErrorRaiserComponents>`. This allows us to implement `ErrorRaiser` for source errors such as `std::io::Error`, `ParseIntError`, and `ErrAuthTokenHasExpired`.

We can also trace the `ErrorRaiser` implementation for `UseDelegate` and see how errors like `std::io::Error` are handled. First, `UseDelegate` implements `ErrorRaiser` because `MyErrorRaiserComponents` implements `DelegateComponent<std::io::Error>`. From there, we observe that the delegate is `RaiseFrom`, and for the case where `Context::Error` is `anyhow::Error`, a `From` instance exists for converting `std::io::Error` into `anyhow::Error`. Thus, the chain of dependencies is satisfied, and `ErrorRaiser` is implemented successfully.

As seen above, the `DelegateComponent` and `delegate_components!` constructs are not only useful for wiring up CGP providers but can also be used to dispatch providers based on the generic parameters of specific traits. In fact, we will see the same pattern applied in other contexts throughout CGP.

For this reason, the `UseDelegate` type is included in the `cgp` crate, along with the `ErrorRaiser` implementation, so that readers can easily identify when delegation is being used every time they encounter a trait implemented for `UseDelegate`.

Forwarding Error Raiser

In addition to the delegation pattern, it can be useful to implement generic error raisers that perform a transformation on the source error and then forward the handling to another error raiser. For instance, when implementing a generic error raiser that formats the source error using `Debug`, we could first format it as a string and then forward the

handling as follows:

```
use cgp::prelude::*;
use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent};
use core::fmt::Debug;

#[cgp_new_provider]
impl<Context, SourceError> ErrorRaiser<Context, SourceError> for DebugError
where
    Context: CanRaiseError<String>,
    SourceError: Debug,
{
    fn raise_error(e: SourceError) -> Context::Error {
        Context::raise_error(format!("{e:?}"))
    }
}
```

In the example above, we define a generic error raiser `DebugError` that implements `ErrorRaiser` for any `SourceError` that implements `Debug`. Additionally, we require that `Context` also implements `CanRaiseError<String>`. Inside the implementation of `raise_error`, we format the source error as a string and then invoke `Context::raise_error` with the formatted string.

A forwarding error raiser like `DebugError` is designed to be used with `UseDelegate`, ensuring that the `ErrorRaiser` implementation for `String` is handled by a separate error raiser. Without this, an incorrect wiring could result in a stack overflow if `DebugError` were to call itself recursively when handling the `String` error.

The key advantage of this approach is that it remains generic over the abstract `Context::Error` type. When used correctly, this allows for a large portion of error handling to remain fully context-generic, promoting flexibility and reusability.

Full Example

Now that we have learned how to use `UseDelegate`, we can rewrite the naive error raiser from the beginning of this chapter and use `delegate_components!` to simplify our error handling.

```
pub mod impls {
    use core::convert::Infallible;
    use core::fmt::{Debug, Display};

    use anyhow::anyhow;
    use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent};
    use cgp::prelude::*;

    #[derive(Debug)]
    pub struct ErrAuthTokenHasExpired;

    #[cgp_new_provider]
    impl<Context, Error> ErrorRaiser<Context, Error> for ReturnError
    where
        Context: HasErrorType<Error = Error>,
    {
        fn raise_error(e: Error) -> Error {
            e
        }
    }

    #[cgp_new_provider]
    impl<Context, SourceError> ErrorRaiser<Context, SourceError> for
RaiseFrom
    where
        Context: HasErrorType,
        Context::Error: From<SourceError>,
    {
        fn raise_error(e: SourceError) -> Context::Error {
            e.into()
        }
    }

    #[cgp_new_provider]
    impl<Context> ErrorRaiser<Context, Infallible> for RaiseInfallible
    where
        Context: HasErrorType,
    {
        fn raise_error(e: Infallible) -> Context::Error {
            match e {}
        }
    }

    #[cgp_new_provider]
    impl<Context, SourceError> ErrorRaiser<Context, SourceError> for
DebugError
    where
        Context: CanRaiseError<String>,
        SourceError: Debug,
    {
        fn raise_error(e: SourceError) -> Context::Error {
            Context::raise_error(format!("{e:?}"))
        }
    }

    #[cgp_new_provider]
```

```
    impl<Context, SourceError> ErrorRaiser<Context, SourceError> for
DisplayAnyhowError
    where
        Context: HasErrorType<Error = anyhow::Error>,
        SourceError: Display,
    {
        fn raise_error(e: SourceError) -> anyhow::Error {
            anyhow!("{e}")
        }
    }
}

pub mod contexts {
    use core::convert::Infallible;
    use core::num::ParseIntError;

    use cgp::core::component::UseDelegate;
    use cgp::core::error::{ErrorRaiserComponent, ErrorTypeProviderComponent};
    use cgp::prelude::*;

    use super::impls::*;

    #[cgp_context]
    pub struct MyApp;

    pub struct MyErrorRaiserComponents;

    delegate_components! {
        MyAppComponents {
            ErrorTypeProviderComponent: UseType<anyhow::Error>,
            ErrorRaiserComponent: UseDelegate<MyErrorRaiserComponents>,
        }
    }

    delegate_components! {
        MyErrorRaiserComponents {
            anyhow::Error: ReturnError,
            Infallible: RaiseInfallible,
            [
                std::io::Error,
                ParseIntError,
            ]:
                RaiseFrom,
            [
                ErrAuthTokenHasExpired,
            ]:
                DebugError,
            [
                String,
                <'a> &'a str,
            ]:
                DisplayAnyhowError,
        }
    }

    pub trait CanRaiseMyAppErrors:
        CanRaiseError<anyhow::Error>
```

```
            + CanRaiseError<Infallible>
            + CanRaiseError<std::io::Error>
            + CanRaiseError<ParseIntError>
            + CanRaiseError<ErrAuthTokenHasExpired>
            + CanRaiseError<String>
            + for<'a> CanRaiseError<&'a str>
        {
        }

        impl CanRaiseMyAppErrors for MyApp {}
    }
```

In the first part of the example, we define various context-generic error raisers that are useful not only for our specific application but can also be reused later for other applications. We have `ReturnError`, which simply returns the source error as-is, `RaiseFrom` for converting the source error using `From`, `RaiseInfallible` for handling `Infallible` errors, and `DebugError` for formatting and re-raising the error as a string. We also define `UseAnyhow` to implement `ProvideErrorType`, and `DisplayAnyhowError` to convert any `SourceError` implementing `Display` into `anyhow::Error`.

In the second part of the example, we define a dummy context, `MyApp`, to illustrate how it can handle various source errors. We define `MyErrorRaiserComponents` and use `delegate_components!` to map various source error types to the corresponding error raiser providers. We then use `UseDelegate<MyErrorRaiserComponents>` as the provider for `ErrorRaiserComponent`. Finally, we define the trait `CanRaiseMyAppErrors` to verify that all the error raisers are wired correctly.

Wiring Checks

As seen in the example, the use of `UseDelegate` with `ErrorRaiser` acts as a form of top-level error handler for an application. The main difference is that the "handling" of errors is done entirely at compile-time, enabling us to customize how each source error is handled without incurring any runtime performance overhead.

However, it's important to note that the wiring for delegated error raisers is done *lazily*, similar to how CGP provider wiring works. This means that an error could be wired incorrectly, with constraints that are not satisfied, and the issue will only manifest as a compile-time error when the error raiser is used in another provider.

Misconfigured wiring of error raisers can often lead to common CGP errors, especially for beginners. We encourage readers to refer back to the chapter on debugging techniques and utilize check traits to ensure all source errors are wired correctly. It's also helpful to use a forked Rust compiler to display unsatisfied constraints arising from incomplete error raiser implementations.

Conclusion

In this chapter, we explored the `UseDelegate` pattern and how it allows us to declaratively handle error raisers in various ways. This pattern simplifies error handling and can be extended to other problem domains within CGP, as we'll see in future chapters. Additionally, the `UseDelegate` pattern serves as a foundation for more advanced error handling techniques, which will be covered in the next chapter.

Error Reporting

In the previous chapter on error handling, we implemented `AuthTokenValidator` to raise the error string `"auth token has expired"`, when a given auth token has expired. Even after we defined a custom error type `ErrAuthTokenHasExpired`, it is still a dummy struct that has a `Debug` implementation that outputs the same string `"auth token has expired"`. In real world applications, we know that it is good engineering practice to include as much details to an error, so that developers and end users can more easily diagnose the source of the problem. On the other hand, it takes a lot of effort to properly design and show good error messages. When doing initial development, we don't necessary want to spend too much effort on formatting error messages, when we don't even know if the code would survive the initial iteration.

To resolve the dilemma, developers are often forced to choose a comprehensive error library that can do everything from error handling to error reporting. Once the library is chosen, implementation code often becomes tightly coupled with the error library. If there is any detail missing in the error report, it may be challenging to include more details without diving deep into the impementation.

CGP offers better ways to resolve this dilemma, by allowing us to decouple the logic of error handling from actual error reporting. In this chapter, we will go into detail of how we can use CGP to improve the error report to show more information about an expired auth token.

Reporting Errors with Abstract Types

One challenge that CGP introduces is that with abstract types, it may be challenging to produce good error report without knowledge about the underlying type. We can workaround this in a naive way by using impl-side dependencies to require the abstract types `Context::AuthToken` and `Context::Time` to implement `Debug`, and then format them as a string before raising it as an error:

```
#[cgp_new_provider]
impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
where
    Context: HasCurrentTime + CanFetchAuthTokenExpiry + for<'a>
CanRaiseError<String>,
    Context::Time: Debug + Ord,
    Context::AuthToken: Debug,
{
    fn validate_auth_token(
        context: &Context,
        auth_token: &Context::AuthToken,
    ) -> Result<(), Context::Error> {
        let now = context.current_time()?;

        let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

        if token_expiry < now {
            Ok(())
        } else {
            Err(Context::raise_error(
                format!(
                    "the auth token {:?} has expired at {:?}, which is
earlier than the current time {:?}",
                    auth_token, token_expiry, now,
                )))
        }
    }
}
```

The example above now shows better error message. But our provider `ValidateTokenIsNotExpired` is now tightly coupled with how the token expiry error is reported. We are now forced to implement `Debug` for any `AuthToken` and `Time` types that we want to use. It is also not possible to customize the error report to instead use the `Display` instance, without directly modifying the implementation for `ValidateTokenIsNotExpired`. Similarly, we cannot easily customize how the message content is formatted, or add additional details to the report.

Source Error Types with Abstract Fields

To better report the error message, we would first re-introduce the `ErrAuthTokenHasExpired` source error type that we have used in earlier examples. But now, we would also add fields with *abstract types* into the struct, so that it contains all values that may be essential for generating a good error report:

```
pub struct ErrAuthTokenHasExpired<'a, Context>
where
    Context: HasAuthTokenType + HasTimeType,
{
    pub context: &'a Context,
    pub auth_token: &'a Context::AuthToken,
    pub current_time: &'a Context::Time,
    pub expiry_time: &'a Context::Time,
}
```

The `ErrAuthTokenHasExpired` struct is now parameterized by a generic lifetime `'a` and a generic context `Context`. Inside the struct, all fields are in the form of reference `&'a`, so that we don't perform any copy to construct the error value. The struct has a `where` clause to require `Context` to implement `HasAuthTokenType` and `HasTimeType`, since we need to hold their values inside the struct. In addition to `auth_token`, `current_time`, and `expiry_time`, we also include a `context` field with a reference to the main context, so that additional error details may be provided through `Context`.

In addition to the struct, we also manually implement a `Debug` instance as a default way to format `ErrAuthTokenHasExpired` as string:

```
impl<'a, Context> Debug for ErrAuthTokenHasExpired<'a, Context>
where
    Context: HasAuthTokenType + HasTimeType,
    Context::AuthToken: Debug,
    Context::Time: Debug,
{
    fn fmt(&self, f: &mut core::fmt::Formatter<'_>) -> core::fmt::Result {
        write!(
            f,
            "the auth token {:?} has expired at {:?}, which is earlier than
the current time {:?}",
            self.auth_token, self.expiry_time, self.current_time,
        )
    }
}
```

Inside the `Debug` instance for `ErrAuthTokenHasExpired`, we make use of impl-side dependencies to require `Context::AuthToken` and `Context::Time` to implement `Debug`. We then use `Debug` to format the values and show the error message.

Notice that even though `ErrAuthTokenHasExpired` contains a `context` field, it is not used in the `Debug` implementation. Also, since the `Debug` constraint for `Context::AuthToken` and `Context::Time` are only present in the `Debug` implementation, it is possible for the concrete types to not implement `Debug`, *if* the application do not use `Debug` with `ErrAuthTokenHasExpired`.

This design is intentional, as we only provide the `Debug` implementation as a *convenience*

for quickly formatting the error message without further customization. On the other hand, a better error reporting strategy may be present elsewhere and provided by the application. The main purpose of this design is so that at the time `ErrAuthTokenHasExpired` and `ValidateTokenIsNotExpired` are defined, we don't need to concern about where and how this error reporting strategy is implemented.

Using the new `ErrAuthTokenHasExpired`, we can now re-implement `ValidateTokenIsNotExpired` as follows:

```
#[cgp_new_provider]
impl<Context> AuthTokenValidator<Context> for ValidateTokenIsNotExpired
where
    Context: HasCurrentTime
        + CanFetchAuthTokenExpiry
        + for<'a> CanRaiseError<ErrAuthTokenHasExpired<'a, Context>>,
    Context::Time: Ord,
{
    fn validate_auth_token(
        context: &Context,
        auth_token: &Context::AuthToken,
    ) -> Result<(), Context::Error> {
        let now = context.current_time()?;

        let token_expiry = context.fetch_auth_token_expiry(auth_token)?;

        if token_expiry < now {
            Ok(())
        } else {
            Err(Context::raise_error(ErrAuthTokenHasExpired {
                context,
                auth_token,
                current_time: &now,
                expiry_time: &token_expiry,
            }))
        }
    }
}
```

In the new implementation, we include the constraint `for<'a> CanRaiseError<ErrAuthTokenHasExpired<'a, Context>>` with *higher ranked trait bound*, so that we can raise `ErrAuthTokenHasExpired` parameterized with any lifetime. Notice that inside the `where` constraints, we no longer require the `Debug` bound on `Context::AuthToken` and `Context::Time`.

With this approach, we have made use of `ErrAuthTokenHasExpired` to fully decouple `ValidateTokenIsNotExpired` provider from the problem of how to report the token expiry error.

Error Report Raisers

In the previous chapter, we have learned about how to define custom error raisers and then dispatch them using the `UseDelegate` pattern. With that in mind, we can easily define error raisers for `ErrAuthTokenHasExpired` to format it in different ways.

One thing to note is that since `ErrAuthTokenHasExpired` contains a lifetime parameter with borrowed values, any error raiser that handles it would likely have to make use of the borrowed value to construct an owned value for `Context::Error`.

The simplest way to raise `ErrAuthTokenHasExpired` is to make use of its `Debug` implementation to and raise it using `DebugError`:

```
use cgp::prelude::*;
use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent};
use core::fmt::Debug;

#[cgp_new_provider]
impl<Context, SourceError> ErrorRaiser<Context, SourceError> for DebugError
where
    Context: CanRaiseError<String>,
    SourceError: Debug,
{
    fn raise_error(e: SourceError) -> Context::Error {
        Context::raise_error(format!("{e:?}"))
    }
}
```

As we discussed in the previous chapter, `DebugError` would implement `ErrorRaiser` if `ErrAuthTokenHasExpired` implements `Debug`. But recall that the `Debug` implementation for `ErrAuthTokenHasExpired` requires both `Context::AuthToken` and `Context::Time` to implement `Debug`. So in a way, the use of impl-side dependencies here is *deeply nested*, but nevertheless still works thanks to Rust's trait system.

Now supposed that instead of using `Debug`, we want to use the `Display` instance of `Context::AuthToken` and `Context::Time` to format the error. Even if we are in a crate that do not own `ErrAuthTokenHasExpired`, we can still implement a custom `ErrorRaiser` instance as follows:

```
#[cgp_new_provider]
impl<'a, Context> ErrorRaiser<Context, ErrAuthTokenHasExpired<'a, Context>>
    for DisplayAuthTokenExpiredError
where
    Context: HasAuthTokenType + HasTimeType + CanRaiseError<String>,
    Context::AuthToken: Display,
    Context::Time: Display,
{
    fn raise_error(e: ErrAuthTokenHasExpired<'a, Context>) -> Context::Error
{
        Context::raise_error(format!(
            "the auth token {} has expired at {}, which is earlier than the
current time {}",
            e.auth_token, e.expiry_time, e.current_time,
        ))
    }
}
```

With this approach, we can now use `DisplayAuthTokenExpiredError` if `Context::AuthToken` and `Context::Time` implement `Display`. But even if they don't, we are still free to choose alternative strategies for our application.

One possible way to improve the error message is to obfuscate the auth token, so that the reader of the error message cannot know about the actual auth token. This may have already been done, if the concrete `AuthToken` type implements a custom `Display` that does so. But in case if it does not, we can still do something similar using a customized error raiser:

```
use sha1::{Digest, Sha1};

#[cgp_new_provider]
impl<'a, Context> ErrorRaiser<Context, ErrAuthTokenHasExpired<'a, Context>>
    for ShowAuthTokenExpiredError
where
    Context: HasAuthTokenType + HasTimeType + CanRaiseError<String>,
    Context::AuthToken: Display,
    Context::Time: Display,
{
    fn raise_error(e: ErrAuthTokenHasExpired<'a, Context>) -> Context::Error
{
        let auth_token_hash =
Sha1::new_with_prefix(e.auth_token.to_string()).finalize();

        Context::raise_error(format!(
            "the auth token {:x} has expired at {}, which is earlier than the
current time {}",
            auth_token_hash, e.expiry_time, e.current_time,
        ))
    }
}
```

By decoupling the error reporting from the provider, we can now customize the error

reporting as we see fit, without needing to access or modify the original provider `ValidateTokenIsNotExpired`.

Context-Specific Error Details

Previously, we included the `context` field in `ErrAuthTokenHasExpired` but never used it in the error reporting. But with the ability to define custom error raisers, we can also define one that extracts additional details from the context, so that it can be included in the error message.

Supposed that we are using `CanValidateAuthToken` in an application that serves sensitive documents. When an expired auth token is used, we may want to also include the document ID being accessed, so that we can identify the attack patterns of any potential attacker. If the application context holds the document ID, we can now access it within the error raiser as follows:

```
use sha1::{Digest, Sha1};

#[cgp_component(DocumentIdGetter)]
pub trait HasDocumentId {
    fn document_id(&self) -> u64;
}

#[cgp_new_provider]
impl<'a, Context> ErrorRaiser<Context, ErrAuthTokenHasExpired<'a, Context>>
    for ShowAuthTokenExpiredError
where
    Context: HasAuthTokenType + HasTimeType + CanRaiseError<String> +
HasDocumentId,
    Context::AuthToken: Display,
    Context::Time: Display,
{
    fn raise_error(e: ErrAuthTokenHasExpired<'a, Context>) -> Context::Error
{
        let document_id = e.context.document_id();
        let auth_token_hash =
Sha1::new_with_prefix(e.auth_token.to_string()).finalize();

        Context::raise_error(format!(
            "failed to access highly sensitive document {} at time {}, using
the auth token {:x} which was expired at {}",
            document_id, e.current_time, auth_token_hash, e.expiry_time,
        ))
    }
}
```

With this, even though the provider `ValidateTokenIsNotExpired` did not know that `Context` contains a document ID, by including the `context` value in

`ErrAuthTokenHasExpired`, we can still implement a custom error raiser that produce a custom error message that includes the document ID.

Conclusion

In this chapter, we have learned about some advanced CGP techniques that can be used to decouple providers from the burden of producing good error reports. With that, we are able to define custom error raisers that produce highly detailed error reports, without needing to modify the original provider implementation. The use of source error types with abstract fields and borrowed values serves as a cheap interface to decouple the producer of an error (the provider) from the handler of an error (the error raiser).

Still, even with CGP, learning all the best practices of properly raising and handling errors can be overwhelming, especially for beginners. Furthermore, even if we can decouple and customize the handling of all possible error cases, extra effort is still needed for every customization, which can still takes a lot of time.

As a result, we do not encourage readers to try and define custom error structs for all possible errors. Instead, readers should start with simple error types like strings, and slowly add more structures to common errors that occur in the application. But readers should keep in mind the techniques introduced in this chapter, so that by the time we need to customize and produce good error reports for our applications, we know about how this can be done using CGP.

Error Wrapping

When programming in Rust, there is a common need to not only raise new errors, but also attach additional details to an error that has previously been raised. This is mainly to allow a caller to attach additional details about which higher-level operations are being performed, so that better error report and diagnostics can be presented to the user.

Error libraries such as `anyhow` and `eyre` provide methods such as `context` and `wrap_err` to allow wrapping of additional details to their error type. In this chapter, we will discuss about how to implement context-generic error wrapping with CGP, and how to integrate them with existing error libraries.

Example: Config Loader

Supposed that we want to build an application with the functionality to load and parse some application configuration from a config path. Using the CGP patterns that we have learned so far, we may implement a context-generic config loader as follows:

```
pub mod traits {
    use std::path::PathBuf;

    use cgp::prelude::*;

    #[cgp_type]
    pub trait HasConfigType {
        type Config;
    }

    #[cgp_component(ConfigLoader)]
    pub trait CanLoadConfig: HasConfigType + HasErrorType {
        fn load_config(&self) -> Result<Self::Config, Self::Error>;
    }

    #[cgp_component(ConfigPathGetter)]
    pub trait HasConfigPath {
        fn config_path(&self) -> &PathBuf;
    }
}

pub mod impls {
    use std::{fs, io};

    use cgp::core::error::{ErrorRaiser, ErrorRaiserComponent};
    use cgp::prelude::*;
    use serde::Deserialize;

    use super::traits::*;

    #[cgp_new_provider]
    impl<Context> ConfigLoader<Context> for LoadJsonConfig
    where
        Context: HasConfigType
            + HasConfigPath
            + CanRaiseError<io::Error>
            + CanRaiseError<serde_json::Error>,
        Context::Config: for<'a> Deserialize<'a>,
    {
        fn load_config(context: &Context) -> Result<Context::Config,
Context::Error> {
            let config_path = context.config_path();

            let config_bytes =
fs::read(config_path).map_err(Context::raise_error)?;

            let config =
serde_json::from_slice(&config_bytes).map_err(Context::raise_error)?;

            Ok(config)
        }
    }
}
```

We first define the `HasConfigType` trait, which provides an abstract `Config` type to

represent the application's config. We then define a `CanLoadConfig` trait, which provides an interface for loading the application config. To help with the implementation, we also implement a `HasConfigPath` trait, which allows a provider to get the file path to the config file from the context.

Using the config traits, we then implement `LoadJsonConfig` as a context-generic provider for `ConfigLoader`, which would read a JSON config file as bytes from the filesystem using `std::fs::read`, and then parse the config using `serde_json`. With CGP, `LoadJsonConfig` can work with any `Config` type that implements `Deserialize`.

We can then define an example application context that makes use of `LoadJsonConfig` to load its config as follows:

```
pub mod contexts {
    use std::io;
    use std::path::PathBuf;

    use cgp::core::component::UseDelegate;
    use cgp::core::error::{ErrorRaiserComponent, ErrorTypeProviderComponent};
    use cgp::prelude::*;
    use serde::Deserialize;

    use super::impls::*;
    use super::traits::*;

    #[cgp_context]
    pub struct App {
        pub config_path: PathBuf,
    }

    #[derive(Deserialize)]
    pub struct AppConfig {
        pub api_secret: String,
    }

    delegate_components! {
        AppComponents {
            ErrorTypeProviderComponent: UseType<anyhow::Error>,
            ErrorRaiserComponent: UseDelegate<RaiseAppErrors>,
            ConfigLoaderComponent: LoadJsonConfig,
        }
    }

    delegate_components! {
        new RaiseAppErrors {
            [
                io::Error,
                serde_json::Error,
            ]:
                RaiseFrom,
        }
    }

    #[cgp_provider]
    impl ConfigTypeProvider<App> for AppComponents {
        type Config = AppConfig;
    }

    #[cgp_provider]
    impl ConfigPathGetter<App> for AppComponents {
        fn config_path(app: &App) -> &PathBuf {
            &app.config_path
        }
    }

    pub trait CanUseApp: CanLoadConfig {}

    impl CanUseApp for App {}
```

```
}
```

The `App` context has a `config_path` field to store the path to the JSON config. We also define an example `AppConfig` type, which implements `Deserialize` and has an `api_secret` string field that can be used by further implementation.

Inside the component wiring for `AppComponents`, we make use of `UseAnyhowError` that we have defined in earlier chapter to provide the `anyhow::Error` type, and we use `UseDelegate<RaiseAppErrors>` to implement the error raiser. Inside of `RaiseAppErrors`, we make use of `RaiseFrom` to convert `std::io::Error` and `serde_json::Error` to `anyhow::Error` using the `From` instance.

We also provide context-specific implementations of `ConfigTypeProvider` and `ConfigPathGetter` for the `App` context. Following that, we define a check trait `CanUseApp` to check that the wiring is done correctly and that `App` implements `CanLoadConfig`.

Even though the example implementation for `LoadJsonConfig` works, we would quickly find out that the error message returned from it is not very helpful. For example, if the file does not exist, we would get the following error message:

```
No such file or directory (os error 2)
```

Similarly, if the config file is not in JSON format, we would get an error message like the following:

```
expected value at line 1 column 2
```

Error messages like above make it very difficult for users to figure out what went wrong, and what action needs to be taken to resolve them. To improve the error messages, we need to *wrap* around source errors like `std::io::Error`, and provide additional details so that the user knows that the error occured when trying to load the app config. Next, we will learn about how to wrap around these errors in CGP.

Error Wrapper

With the same motivation described in the previous chapter, we would like to make use of CGP to also enable modular error reporting for the error details that is being wrapped. This would mean that we want to define a generic `Detail` type that can include *structured data* inside the error details. We can do that by introduce an *error wrapper* trait as follows:

```rust
#[cgp_component(ErrorWrapper)]
pub trait CanWrapError<Detail>: HasErrorType {
    fn wrap_error(error: Self::Error, detail: Detail) -> Self::Error;
}
```

The `CanWrapError` trait is parameterized by a generic `Detail` type, and has `HasErrorType` as its supertrait. Inside the `wrap_error` method, it first accepts a context error `Self::Error` and also a `Detail` value. It then wraps the detail inside the context error, and return `Self::Error`.

To see how `CanWrapError` works in practice, we can redefine `LoadJsonConfig` to use `CanWrapError` as follows:

```rust
#[cgp_new_provider]
impl<Context> ConfigLoader<Context> for LoadJsonConfig
where
    Context: HasConfigType
        + HasConfigPath
        + CanWrapError<String>
        + CanRaiseError<io::Error>
        + CanRaiseError<serde_json::Error>,
    Context::Config: for<'a> Deserialize<'a>,
{
    fn load_config(context: &Context) -> Result<Context::Config,
Context::Error> {
        let config_path = context.config_path();

        let config_bytes = fs::read(config_path).map_err(|e| {
            Context::wrap_error(
                Context::raise_error(e),
                format!(
                    "error when reading config file at path {}",
                    config_path.display()
                ),
            )
        })?;

        let config = serde_json::from_slice(&config_bytes).map_err(|e| {
            Context::wrap_error(
                Context::raise_error(e),
                format!(
                    "error when parsing JSON config file at path {}",
                    config_path.display()
                ),
            )
        })?;

        Ok(config)
    }
}
```

Inside the new implementation of `LoadJsonConfig`, we add a `CanWrapError<String>`

constraint so that we can add stringly error details inside the provider. When mapping the errors returned from `std::fs::read` and `serde_json::from_slice`, we pass in a closure instead of directly calling `Context::raise_error`. Since the first argument of `wrap_error` expects a `Context::Error`, we would still first use `Context::raise_error` to raise `std::io::Error` and `serde_json::Error` into `Context::Error`. In the second argument, we use `format!` to add additional details that the errors occured when we are trying to read and parse the given config file.

By looking only at the example, it may seem redundant that we have to first raise a concrete source error like `std::io::Error` into `Context::Error`, before wrapping it again using `Context::wrap_error`. If the reader prefers, you can also use a constraint like `CanRaiseError<(String, std::io::Error)>` to raise the I/O error with additional string detail.

However, the interface for `CanWrapError` is more applicable generally, especially when we combine the use with other abstractions. For example, we may want to define a trait like `CanReadFile` to try reading a file, and returning a general `Context::Error` when the read fails. In that case, we can still use `wrap_error` without knowing about whether we are dealing with concrete errors or abstract errors.

Next, we would need to implement a provider for `CanWrapError` to handle how to wrap additional details into the error value. In the case when the context error type is `anyhow::Error`, we can simply call the `context` method. So we can implement an error wrapper provider for `anyhow::Error` as follows:

```
#[cgp_new_provider]
impl<Context, Detail> ErrorWrapper<Context, Detail> for WrapWithAnyhowContext
where
    Context: HasErrorType<Error = anyhow::Error>,
    Detail: Display + Send + Sync + 'static,
{
    fn wrap_error(error: anyhow::Error, detail: Detail) -> anyhow::Error {
        error.context(detail)
    }
}
```

We implement `WrapWithAnyhowContext` as a context-generic provider for `anyhow::Error`. It is implemented for any context type `Context` with `Context::Error` being the same as `anyhow::Error`. Additionally, it is implemented for any `Detail` type that implements `Display + Send + Sync + 'static`, as those are the required trait bounds to use `anyhow::Error::context`. Inside the `wrap_error` implementation, we simply call `error.context(detail)` to wrap the error detail using `anyhow`.

After rewiring the application with the new providers, if we run the application again with missing file, it would show the following error instead:

```
error when reading config file at path config.json

Caused by:
    No such file or directory (os error 2)
```

Similarly, when encountering error parsing the config JSON, the application now shows the error message:

```
error when parsing JSON config file at path config.toml

Caused by:
    expected value at line 1 column 2
```

As we can see, the error messages are now much more informative, allowing the user to diagnose what went wrong and fix the problem.

Structured Error Wrapping

Similar to the reasons for using structured error reporting from the previous chapter, using structured error details would make it possible to decouple how to format the wrapped error detail from the provider. For the case of `LoadJsonConfig`, we can define and use a structured error detail type as follows:

```
pub struct ErrLoadJsonConfig<'a, Context> {
    pub context: &'a Context,
    pub config_path: &'a PathBuf,
    pub action: LoadJsonConfigAction,
}

pub enum LoadJsonConfigAction {
    ReadFile,
    ParseFile,
}

#[cgp_new_provider]
impl<Context> ConfigLoader<Context> for LoadJsonConfig
where
    Context: HasConfigType
        + HasConfigPath
        + CanRaiseError<io::Error>
        + CanRaiseError<serde_json::Error>
        + for<'a> CanWrapError<ErrLoadJsonConfig<'a, Context>>,
    Context::Config: for<'a> Deserialize<'a>,
{
    fn load_config(context: &Context) -> Result<Context::Config,
Context::Error> {
        let config_path = context.config_path();

        let config_bytes = fs::read(config_path).map_err(|e| {
            Context::wrap_error(
                Context::raise_error(e),
                ErrLoadJsonConfig {
                    context,
                    config_path,
                    action: LoadJsonConfigAction::ReadFile,
                },
            )
        })?;

        let config = serde_json::from_slice(&config_bytes).map_err(|e| {
            Context::wrap_error(
                Context::raise_error(e),
                ErrLoadJsonConfig {
                    context,
                    config_path,
                    action: LoadJsonConfigAction::ParseFile,
                },
            )
        })?;

        Ok(config)
    }
}

impl<'a, Context> Debug for ErrLoadJsonConfig<'a, Context> {
    fn fmt(&self, f: &mut core::fmt::Formatter<'_>) -> core::fmt::Result {
        match self.action {
            LoadJsonConfigAction::ReadFile => {
                write!(
```

```
                        f,
                        "error when reading config file at path {}",
                        self.config_path.display()
                    )
                }
                LoadJsonConfigAction::ParseFile => {
                    write!(
                        f,
                        "error when parsing JSON config file at path {}",
                        self.config_path.display()
                    )
                }
            }
        }
    }
```

We first define an error detail struct `ErrLoadJsonConfig` that is parameterized by a lifetime `'a` and a context type `Context`. Inside the struct, we include the `Context` field to allow potential extra details to be included from the concrete context. We also include the `config_path` to show the path of the config file that cause the error. Lastly, we also include a `LoadJsonConfigAction` field to indicate whether the error happened when reading or parsing the config file.

We also implement a `Debug` instance for `ErrLoadJsonConfig`, so that it can be used by default when there is no need to customize the display of the error detail. The `Debug` implementation ignores the `context` field, and shows the same error messages as we did before.

To make use of the `Debug` implementation with `anyhow`, we can implement a separate provider that wraps any `Detail` type that implements `Debug` as follows:

```
#[cgp_new_provider]
impl<Context, Detail> ErrorWrapper<Context, Detail> for WrapWithAnyhowDebug
where
    Context: HasErrorType<Error = anyhow::Error>,
    Detail: Debug,
{
    fn wrap_error(error: anyhow::Error, detail: Detail) -> anyhow::Error {
        error.context(format!("{detail:?}"))
    }
}
```

To wrap the error, we first use `Debug` to format the error detail into string, and then call `error.context` with the string.

Full Example

With everything that we have learned so far, we can rewrite the config loader example in the beginning of this chapter, and make use of `CanWrapError` to decouple the error wrapping details from the provider `LoadJsonConfig`:

```
pub mod traits {
    use std::path::PathBuf;

    use cgp::core::component::UseDelegate;
    use cgp::prelude::*;

    #[cgp_type]
    pub trait HasConfigType {
        type Config;
    }

    #[cgp_component(ConfigLoader)]
    pub trait CanLoadConfig: HasConfigType + HasErrorType {
        fn load_config(&self) -> Result<Self::Config, Self::Error>;
    }

    #[cgp_component(ConfigPathGetter)]
    pub trait HasConfigPath {
        fn config_path(&self) -> &PathBuf;
    }
}

pub mod impls {
    use core::fmt::{Debug, Display};
    use std::path::PathBuf;
    use std::{fs, io};

    use cgp::core::error::{ErrorRaiser, ErrorWrapper, ErrorRaiserComponent,
ErrorWrapperComponent};
    use cgp::prelude::*;
    use serde::Deserialize;

    use super::traits::*;

    pub struct ErrLoadJsonConfig<'a, Context> {
        pub context: &'a Context,
        pub config_path: &'a PathBuf,
        pub action: LoadJsonConfigAction,
    }

    pub enum LoadJsonConfigAction {
        ReadFile,
        ParseFile,
    }

    #[cgp_new_provider]
    impl<Context> ConfigLoader<Context> for LoadJsonConfig
    where
        Context: HasConfigType
            + HasConfigPath
            + CanRaiseError<io::Error>
            + CanRaiseError<serde_json::Error>
            + for<'a> CanWrapError<ErrLoadJsonConfig<'a, Context>>,
        Context::Config: for<'a> Deserialize<'a>,
    {
        fn load_config(context: &Context) -> Result<Context::Config,
```

```
Context::Error> {
            let config_path = context.config_path();

            let config_bytes = fs::read(config_path).map_err(|e| {
                Context::wrap_error(
                    Context::raise_error(e),
                    ErrLoadJsonConfig {
                        context,
                        config_path,
                        action: LoadJsonConfigAction::ReadFile,
                    },
                )
            })?;

            let config = serde_json::from_slice(&config_bytes).map_err(|e| {
                Context::wrap_error(
                    Context::raise_error(e),
                    ErrLoadJsonConfig {
                        context,
                        config_path,
                        action: LoadJsonConfigAction::ParseFile,
                    },
                )
            })?;

            Ok(config)
        }
    }

    impl<'a, Context> Debug for ErrLoadJsonConfig<'a, Context> {
        fn fmt(&self, f: &mut core::fmt::Formatter<'_>) -> core::fmt::Result
{
            match self.action {
                LoadJsonConfigAction::ReadFile => {
                    write!(
                        f,
                        "error when reading config file at path {}",
                        self.config_path.display()
                    )
                }
                LoadJsonConfigAction::ParseFile => {
                    write!(
                        f,
                        "error when parsing JSON config file at path {}",
                        self.config_path.display()
                    )
                }
            }
        }
    }

    #[cgp_new_provider]
    impl<Context, SourceError> ErrorRaiser<Context, SourceError> for
RaiseFrom
    where
        Context: HasErrorType,
        Context::Error: From<SourceError>,
```

```
    {
        fn raise_error(e: SourceError) -> Context::Error {
            e.into()
        }
    }

    #[cgp_new_provider]
    impl<Context, Detail> ErrorWrapper<Context, Detail> for
WrapWithAnyhowDebug
    where
        Context: HasErrorType<Error = anyhow::Error>,
        Detail: Debug,
    {
        fn wrap_error(error: anyhow::Error, detail: Detail) -> anyhow::Error
{
            error.context(format!("{detail:?}"))
        }
    }
}

pub mod contexts {
    use std::io;
    use std::path::PathBuf;

    use cgp::core::component::UseDelegate;
    use cgp::core::error::{ErrorRaiserComponent, ErrorWrapperComponent,
ErrorTypeProviderComponent};
    use cgp::prelude::*;
    use serde::Deserialize;

    use super::impls::*;
    use super::traits::*;

    #[cgp_context]
    pub struct App {
        pub config_path: PathBuf,
    }

    #[derive(Deserialize)]
    pub struct AppConfig {
        pub secret: String,
    }

    pub struct RaiseAppErrors;

    delegate_components! {
        AppComponents {
            ErrorTypeProviderComponent: UseType<anyhow::Error>,
            ErrorRaiserComponent: UseDelegate<RaiseAppErrors>,
            ErrorWrapperComponent: WrapWithAnyhowDebug,
            ConfigLoaderComponent: LoadJsonConfig,
        }
    }

    delegate_components! {
        RaiseAppErrors {
            [
```

```
                io::Error,
                serde_json::Error,
            ]:
                RaiseFrom,
        }
    }

    #[cgp_provider]
    impl ConfigTypeProvider<App> for AppComponents {
        type Config = AppConfig;
    }

    #[cgp_provider]
    impl ConfigPathGetter<App> for AppComponents {
        fn config_path(app: &App) -> &PathBuf {
            &app.config_path
        }
    }

    pub trait CanUseApp: CanLoadConfig {}

    impl CanUseApp for App {}
}
```

Delegated Error Wrapping

Similar to the previous chapter on delegated error raisers, we can also make use of the `UseDelegate` pattern to implement delegated error wrapping as follows:

```
#[cgp_provider]
impl<Context, Detail, Components> ErrorWrapper<Context, Detail> for
UseDelegate<Components>
where
    Context: HasErrorType,
    Components: DelegateComponent<Detail>,
    Components::Delegate: ErrorWrapper<Context, Detail>,
{
    fn wrap_error(error: Context::Error, detail: Detail) -> Context::Error {
        Components::Delegate::wrap_error(error, detail)
    }
}
```

With this implementation, we can dispatch the handling of different error `Detail` type to different error wrappers, similar to how we dispatch the error raisers based on the `SourceError` type:

```rust
#[cgp_context]
pub struct App {
    pub config_path: PathBuf,
}

#[derive(Deserialize)]
pub struct AppConfig {
    pub secret: String,
}

pub struct RaiseAppErrors;

pub struct WrapAppErrors;

delegate_components! {
    AppComponents {
        ErrorTypeProviderComponent: UseType<anyhow::Error>,
        ErrorRaiserComponent: UseDelegate<RaiseAppErrors>,
        ErrorWrapperComponent: UseDelegate<WrapAppErrors>,
        ConfigLoaderComponent: LoadJsonConfig,
    }
}

delegate_components! {
    RaiseAppErrors {
        [
            io::Error,
            serde_json::Error,
        ]:
            RaiseFrom,
    }
}

delegate_components! {
    WrapAppErrors {
        String: WrapWithAnyhowContext,
        <'a, Context> ErrLoadJsonConfig<'a, Context>:
            WrapWithAnyhowDebug,
        // add other error wrappers here
    }
}
```

The above example shows the addition of a new `WrapAppErrors` type, which we use with `delegate_components!` to map the handling of `String` detail to `WrapWithAnyhowContext`, and `ErrLoadJsonConfig` detail to `WrapWithAnyhowDebug`. Following the same pattern, we will be able to customize how exactly each error detail is wrapped, by updating the mapping for `WrapAppErrors`.

Conclusion

In this chapter, we learned about how to perform abstract error wrapping to wrap additional details to an abstract error. The pattern for using `CanWrapError` is very similar to the patterns that we have previously learned for `CanRaiseError`. So this is mostly a recap of the same patterns, and also show readers how you can expect the same CGP pattern to be applied in many different places.

Similar to the advice from the previous chapters, it could be overwhelming for beginners to try to use the full structured error wrapping patterns introduced in this chapter. As a result, we encourage readers to start with using only `String` as the error detail when wrapping errors inside practice applications.

The need for structured error wrapping typically would only arise in large-scale applications, or when one wants to publish CGP-based library crates for others to build modular applications. As such, you can always revisit this chapter at a later time, and refactor your providers to make use of structured error details when you really need them.

Field Accessors

With impl-side dependencies, CGP offers a way to inject dependencies into providers without cluttering the public interfaces with extra constraints. One common use of this dependency injection is for a provider to retrieve values from the context. This pattern is often referred to as a field *accessor* or *getter*, since it involves accessing field values from the context. In this chapter, we'll explore how to define and use field accessors effectively with CGP.

Example: API Call

Suppose our application needs to make API calls to an external service to read messages by their message ID. To abstract away the details of the API call, we can define CGP traits as follows:

```
use cgp::prelude::*;

#[cgp_type]
pub trait HasMessageType {
    type Message;
}

#[cgp_type]
pub trait HasMessageIdType {
    type MessageId;
}

#[cgp_component(MessageQuerier)]
pub trait CanQueryMessage: HasMessageIdType + HasMessageType + HasErrorType {
    fn query_message(&self, message_id: &Self::MessageId) ->
Result<Self::Message, Self::Error>;
}
```

Following the patterns for associated types, we define the type traits `HasMessageIdType` and `HasMessageType` to abstract away the details of the message ID and message structures. Additionally, the `CanQueryMessage` trait accepts an abstract `MessageId` and returns either an abstract `Message` or an abstract `Error`, following the patterns for error handling.

With the interfaces defined, we now implement a simple API client provider that queries the message via an HTTP request.

```rust
use reqwest::blocking::Client;
use reqwest::StatusCode;
use serde::Deserialize;
#[derive(Debug)]
pub struct ErrStatusCode {
    pub status_code: StatusCode,
}

#[derive(Deserialize)]
pub struct ApiMessageResponse {
    pub message: String,
}

#[cgp_new_provider]
impl<Context> MessageQuerier<Context> for ReadMessageFromApi
where
    Context: HasMessageIdType<MessageId = u64>
        + HasMessageType<Message = String>
        + CanRaiseError<reqwest::Error>
        + CanRaiseError<ErrStatusCode>,
{
    fn query_message(_context: &Context, message_id: &u64) -> Result<String,
Context::Error> {
        let client = Client::new();

        let response = client
            .get(format!("http://localhost:8000/api/messages/{message_id}"))
            .send()
            .map_err(Context::raise_error)?;

        let status_code = response.status();

        if !status_code.is_success() {
            return Err(Context::raise_error(ErrStatusCode { status_code }));
        }

        let message_response: ApiMessageResponse =
response.json().map_err(Context::raise_error)?;

        Ok(message_response.message)
    }
}
```

For the purposes of the examples in this chapter, we will use the `reqwest` library to make HTTP calls. We will also use the *blocking* version of the API in this chapter, as asynchronous programming in CGP will be covered in later chapters.

In the example above, we implement `MessageQuerier` for the `ReadMessageFromApi` provider. For simplicity, we add the constraint that `MessageId` must be of type `u64` and the `Message` type is a basic `String`.

We also use the context to handle errors. Specifically, we raise the `reqwest::Error` returned by the `reqwest` methods, as well as a custom `ErrStatusCode` error if the server

responds with an error HTTP status.

Within the method, we first create a `reqwest::Client`, and then use it to send an HTTP GET request to the URL `"http://localhost:8000/api/messages/{message_id}"`. If the returned HTTP status is unsuccessful, we raise the `ErrStatusCode`. Otherwise, we parse the response body as JSON into the `ApiMessageResponse` struct, which expects the response to contain a `message` field.

It's clear that the naive provider has some hard-coded values. For instance, the API base URL `http://localhost:8000` is fixed, but it should be configurable. In the next section, we will explore how to define *accessor* traits to retrieve these configurable values from the context.

Getting the Base API URL

In CGP, defining an accessor trait to retrieve values from the context is straightforward. To make the base API URL configurable, we define a `HasApiBaseUrl` trait as follows:

```
#[cgp_component(ApiBaseUrlGetter)]
pub trait HasApiBaseUrl {
    fn api_base_url(&self) -> &String;
}
```

The `HasApiBaseUrl` trait defines a method, `api_base_url`, which returns a reference to a `String` from the context. In production applications, you might prefer to return a `url::Url` or even an abstract `Url` type instead of a `String`. However, for simplicity, we use a `String` in this example.

Next, we can include the `HasApiBaseUrl` trait within `ReadMessageFromApi`, allowing us to construct the HTTP request using the base API URL provided by the context:

```
#[cgp_new_provider]
impl<Context> MessageQuerier<Context> for ReadMessageFromApi
where
    Context: HasMessageIdType<MessageId = u64>
        + HasMessageType<Message = String>
        + HasApiBaseUrl
        + CanRaiseError<reqwest::Error>
        + CanRaiseError<ErrStatusCode>,
{
    fn query_message(context: &Context, message_id: &u64) -> Result<String,
Context::Error> {
        let client = Client::new();

        let url = format!("{}/api/messages/{}", context.api_base_url(),
message_id);

        let response = client.get(url).send().map_err(Context::raise_error)?;

        let status_code = response.status();

        if !status_code.is_success() {
            return Err(Context::raise_error(ErrStatusCode { status_code }));
        }

        let message_response: ApiMessageResponse =
response.json().map_err(Context::raise_error)?;

        Ok(message_response.message)
    }
}
```

Getting the Auth Token

In addition to the base API URL, many API services require authentication to protect their resources from unauthorized access. For this example, we'll use simple *bearer tokens* for API access.

Just as we did with `HasApiBaseUrl`, we can define a `HasAuthToken` trait to retrieve the authentication token as follows:

```
#[cgp_type]
pub trait HasAuthTokenType {
    type AuthToken;
}

#[cgp_component(AuthTokenGetter)]
pub trait HasAuthToken: HasAuthTokenType {
    fn auth_token(&self) -> &Self::AuthToken;
}
```

Similar to the pattern used in the earlier chapter, we first define `HasAuthTokenType` to keep the `AuthToken` type abstract. In fact, this `HasAuthTokenType` trait and its associated providers can be reused across different chapters or applications. This demonstrates how minimal CGP traits facilitate the reuse of interfaces in multiple contexts.

Next, we define a getter trait, `HasAuthToken`, which provides access to an abstract `AuthToken` value from the context. With this in place, we can now update `ReadMessageFromApi` to include the authentication token in the `Authorization` HTTP header:

```
#[cgp_new_provider]
impl<Context> MessageQuerier<Context> for ReadMessageFromApi
where
    Context: HasMessageIdType<MessageId = u64>
        + HasMessageType<Message = String>
        + HasApiBaseUrl
        + HasAuthToken
        + CanRaiseError<reqwest::Error>
        + CanRaiseError<ErrStatusCode>,
    Context::AuthToken: Display,
{
    fn query_message(context: &Context, message_id: &u64) -> Result<String,
Context::Error> {
        let client = Client::new();

        let url = format!("{}/api/messages/{}", context.api_base_url(),
message_id);

        let response = client
            .get(url)
            .bearer_auth(context.auth_token())
            .send()
            .map_err(Context::raise_error)?;

        let status_code = response.status();

        if !status_code.is_success() {
            return Err(Context::raise_error(ErrStatusCode { status_code }));
        }

        let message_response: ApiMessageResponse =
response.json().map_err(Context::raise_error)?;

        Ok(message_response.message)
    }
}
```

In this updated code, we use the `bearer_auth` method from the `reqwest` library to include the authentication token in the HTTP header. In this case, the provider only requires that `Context::AuthToken` implement the `Display` trait, allowing it to work with custom `AuthToken` types, not limited to `String`.

Traits with Multiple Getter Methods

As you build providers that need access to various pieces of data – like a `ReadMessageFromApi` provider that requires both an `api_base_url` and an `auth_token` – you might initially consider grouping these accessors into a single trait.

Here's an example of how you *could* define a trait with multiple getter methods for API client fields:

```
#[cgp_component(ApiClientFieldsGetter)]
pub trait HasApiClientFields: HasAuthTokenType {
    fn api_base_url(&self) -> &String;

    fn auth_token(&self) -> &Self::AuthToken;
}
```

While this approach is syntactically valid, it introduces unnecessary coupling. If a provider only needs the `api_base_url`, it still inherits a dependency on the `auth_token`. This could potentially impact modularity and reusability.

Furthermore, this design prevents you from defining separate, independent providers for `api_base_url` and `auth_token`, each potentially with its own distinct logic for obtaining that data.

Traits containing only one method also unlock powerful patterns like the `UseField` pattern (which we'll introduce later). This pattern significantly simplifies the boilerplate required to implement accessor traits and enables the creation of reusable accessor providers.

Ultimately, CGP provides the flexibility to design your traits in a way that suits your needs. You are not strictly prevented from defining multiple accessor methods within a single trait, or even mixing getters with associated types or other methods.

However, you may find that defining accessor traits that each contain only one getter method is a frequently used pattern, often chosen for its potential benefits in modularity, reusability, and compatibility with helper patterns like UseField. You'll see this pattern frequently demonstrated throughout this book.

Implementing Accessor Providers

Now that we have implemented the provider, we would look at how to implement a concrete context that uses `ReadMessageFromApi` and implement the accessors. We can implement an `ApiClient` context that makes use of all providers as follows:

```rust
#[cgp_context]
pub struct ApiClient {
    pub api_base_url: String,
    pub auth_token: String,
}

delegate_components! {
    ApiClientComponents {
        ErrorTypeProviderComponent:
            UseAnyhowError,
        ErrorRaiserComponent:
            UseDelegate<RaiseApiErrors>,
        MessageIdTypeProviderComponent:
            UseType<u64>,
        MessageTypeProviderComponent:
            UseType<String>,
        AuthTokenTypeProviderComponent:
            UseType<String>,
        MessageQuerierComponent:
            ReadMessageFromApi,
    }
}

delegate_components! {
    new RaiseApiErrors {
        reqwest::Error: RaiseFrom,
        ErrStatusCode: DebugAnyhowError,
    }
}

#[cgp_provider]
impl ApiBaseUrlGetter<ApiClient> for ApiClientComponents {
    fn api_base_url(api_client: &ApiClient) -> &String {
        &api_client.api_base_url
    }
}

#[cgp_provider]
impl AuthTokenGetter<ApiClient> for ApiClientComponents {
    fn auth_token(api_client: &ApiClient) -> &String {
        &api_client.auth_token
    }
}

pub trait CanUseApiClient: CanQueryMessage {}

impl CanUseApiClient for ApiClient {}
```

The `ApiClient` context is defined with the fields that we need to implement the accessor traits. We then have context-specific implementation of `ApiBaseUrlGetter` and `AuthTokenGetter` to work directly with `ApiClient`. With that, our wiring is completed, and we can check that `ApiClient` implements `CanQueryMessage`.

Context-Generic Accessor Providers

While the previous accessor implementation for `ApiClient` works, it requires explicit and concrete access to the `ApiClient` context to implement the accessors. While this approach is manageable with only a couple of accessor methods, it can quickly become cumbersome as the application grows and requires numerous accessors across multiple contexts. A more efficient approach would be to implement *context-generic* providers for field accessors, allowing us to reuse them across any context that contains the relevant field.

To enable the implementation of context-generic accessors, the `cgp` crate provides a derivable `HasField` trait. This trait acts as a *proxy*, allowing access to fields in a concrete context. The trait is defined as follows:

```
pub trait HasField<Tag> {
    type Value;

    fn get_field(&self, _tag: PhantomData<Tag>) -> &Self::Value;
}
```

For each field within a concrete context, we can implement a `HasField` instance by associating a `Tag` type with the field's *name* and an associated type `Value` representing the field's *type*. Additionally, the `HasField` trait includes a `get_field` method, which retrieves a reference to the field value from the context. The `get_field` method accepts an additional `tag` parameter, which is a `PhantomData` type parameter tied to the field's name `Tag`. This phantom parameter helps with type inference in Rust, as without it, Rust would not be able to deduce which field associated with `Tag` is being accessed.

We can automatically derive `HasField` instances for a context like `ApiClient` using the derive macro, as shown below:

```
#[derive(HasField)]
pub struct ApiClient {
    pub api_base_url: String,
    pub auth_token: String,
}
```

The derive macro would then generate the corresponding `HasField` instances for `ApiClient`:

```
impl HasField<symbol!("api_base_url")> for ApiClient {
    type Value = String;

    fn get_field(&self, _tag: PhantomData<symbol!("api_base_url")>) ->
&String {
        &self.api_base_url
    }
}

impl HasField<symbol!("auth_token")> for ApiClient {
    type Value = String;

    fn get_field(&self, _tag: PhantomData<symbol!("auth_token")>) -> &String
{
        &self.auth_token
    }
}
```

Symbols

In the derived `HasField` instances, we observe the use of `symbol!("api_base_url")` and `symbol!("auth_token")` for the `Tag` generic type. While a string like `"api_base_url"` is a value of type `&str`, we need to use it as a *type* within the `Tag` parameter. To achieve this, we use the `symbol!` macro to "lift" a string value into a unique type, which allows us to treat the string `"api_base_url"` as a *type*. Essentially, this means that if the string content is the same across two uses of `symbol!`, the types will be treated as equivalent.

Behind the scenes, the `symbol!` macro first uses the `Char` type to "lift" individual characters into types. The `Char` type is defined as follows:

```
pub struct Char<const CHAR: char>;
```

This makes use of Rust's *const generics* feature to parameterize `Char` with a constant `CHAR` of type `char`. The `Char` struct itself is empty, as we only use it for type-level manipulation.

Although we can use const generics to lift individual characters, we currently cannot use a type like `String` or `&str` within const generics. As a workaround, we construct a *type-level list* of characters. For example, `symbol!("abc")` is desugared to a type-level list of characters like:

```
(Char<'a'>, (Char<'b'>, (Char<'c'>, ())))
```

In `cgp`, instead of using Rust's native tuple, we define the `Cons` and `Nil` types to represent type-level lists:

```
pub struct Nil;

pub struct Cons<Head, Tail>(pub Head, pub Tail);
```

The `Nil` type represents an empty type-level list, while `Cons` is used to prepend an element to the front of the list, similar to how linked lists work in Lisp.

Thus, the actual desugaring of `symbol!("abc")` looks like this:

```
Cons<Char<'a'>, Cons<Char<'b'>, Cons<Char<'c'>, Nil>>>
```

While this type may seem complex, it has a compact representation from the perspective of the Rust compiler. Furthermore, since we don't construct values from symbol types at runtime, there is no runtime overhead associated with them. The use of `HasField` to implement context-generic accessors introduces negligible compile-time overhead, even in large codebases.

It's important to note that the current representation of symbols is a temporary workaround. Once Rust supports using strings in const generics, we can simplify the desugaring process and adjust our implementation accordingly.

If the explanation here still feels unclear, think of symbols as strings being used as *types* rather than values. In later sections, we'll explore how `cgp` provides additional abstractions that abstract away the use of `symbol!` and `HasField`. These abstractions simplify the process, so you won't need to worry about these details in simple cases.

Auto Accessor Traits

The process of defining and wiring many CGP components can be overwhelming for developers who are new to CGP. In the early stages of a project, there is typically not much need for customizing how fields are accessed. As a result, some developers may find the full use of field accessors introduced in this chapter unnecessarily complex.

To simplify the use of accessor traits, one approach is to define them not as CGP components, but as regular Rust traits with blanket implementations that leverage `HasField`. For example, we can redefine the `HasApiBaseUrl` trait as follows:

```rust
pub trait HasApiBaseUrl {
    fn api_base_url(&self) -> &String;
}

impl<Context> HasApiBaseUrl for Context
where
    Context: HasField<symbol!("api_base_url"), Value = String>,
{
    fn api_base_url(&self) -> &String {
        self.get_field(PhantomData)
    }
}
```

With this approach, the `HasApiBaseUrl` trait will be automatically implemented for any context that derives `HasField` and contains the relevant field. There is no longer need for explicit wiring of the `ApiBaseUrlGetterComponent` within the context components.

This approach allows providers, such as `ReadMessageFromApi`, to still use accessor traits like `HasApiBaseUrl` to simplify field access. Meanwhile, context implementers can simply use `#[derive(HasField)]` without having to worry about manual wiring.

The main drawback of this approach is that the context cannot easily override the implementation of `HasApiBaseUrl`, unless it opts not to implement `HasField`. However, it would be straightforward to refactor the trait in the future to convert it into a full CGP component.

Overall, this approach may be an appealing option for developers who want a simpler experience with CGP without fully utilizing its advanced features.

The `#[cgp_auto_getter]` Macro

To streamline the creation of auto accessor traits, the `cgp` crate provides the `#[cgp_auto_getter]` macro, which derives blanket implementations for accessor traits. For instance, the earlier example can be rewritten as follows:

```rust
#[cgp_auto_getter]
pub trait HasApiBaseUrl {
    fn api_base_url(&self) -> &String;
}

#[cgp_auto_getter]
pub trait HasAuthToken: HasAuthTokenType {
    fn auth_token(&self) -> &Self::AuthToken;
}
```

Since `#[cgp_auto_getter]` generates a blanket implementation leveraging `HasField`

directly, there is no corresponding provider trait being derived in this case.

The `#[cgp_auto_getter]` attribute can also be applied to accessor traits that define multiple getter methods. For instance, we can combine two accessor traits into one, as shown below:

```
#[cgp_auto_getter]
pub trait HasApiClientFields: HasAuthTokenType {
    fn api_base_url(&self) -> &String;

    fn auth_token(&self) -> &Self::AuthToken;
}
```

By using `#[cgp_auto_getter]`, accessor traits are automatically implemented for contexts that use `#[derive(HasField)]` and include fields matching the names and return types of the accessor methods. This approach encapsulates the use of `HasField` and `symbol!`, providing well-typed and idiomatic interfaces for field access while abstracting the underlying mechanics.

Using `HasField` in Accessor Providers

With `HasField`, we can implement context-generic providers like `ApiUrlGetter`. Here's an example:

```
#[cgp_new_provider]
impl<Context> ApiBaseUrlGetter<Context> for GetApiUrl
where
    Context: HasField<symbol!("api_url"), Value = String>,
{
    fn api_base_url(context: &Context) -> &String {
        context.get_field(PhantomData)
    }
}
```

In this implementation, `GetApiUrl` is defined for any `Context` type that implements `HasField<symbol!("api_url"), Value = String>`. This means that as long as the context uses `#[derive(HasField)]`, and has a field named `api_url` of type `String`, the `GetApiUrl` provider can be used with it.

Similarly, we can implement a context-generic provider for `AuthTokenGetter` as follows:

```
#[cgp_new_provider]
impl<Context> AuthTokenGetter<Context> for GetAuthToken
where
    Context: HasAuthTokenType + HasField<symbol!("auth_token"), Value =
Context::AuthToken>,
{
    fn auth_token(context: &Context) -> &Context::AuthToken {
        context.get_field(PhantomData)
    }
}
```

The `GetAuthToken` provider is slightly more complex since the `auth_token` method returns an abstract `Context::AuthToken` type. To handle this, we require the `Context` to implement `HasAuthTokenType` and for the `Value` associated type to match `Context::AuthToken`. This ensures that `GetAuthToken` can be used with any context that has an `auth_token` field of the same type as the `AuthToken` defined in `HasAuthTokenType`.

The `UseFields` Pattern

The providers `GetAuthToken` and `GetApiUrl` share a common characteristic: they implement accessor traits for any context type by utilizing `HasField`, with the field name corresponding to the accessor method name. To streamline this pattern, `cgp` provides the `UseFields` marker struct, which simplifies the implementation of such providers:

```
struct UseFields;
```

With `UseFields`, we can bypass the need to define custom provider structs and implement the logic directly on `UseFields`, as shown below:

```rust
#[cgp_provider]
impl<Context> ApiBaseUrlGetter<Context> for UseFields
where
    Context: HasField<symbol!("api_url"), Value = String>,
{
    fn api_base_url(context: &Context) -> &String {
        context.get_field(PhantomData)
    }
}

#[cgp_provider]
impl<Context> AuthTokenGetter<Context> for UseFields
where
    Context: HasAuthTokenType + HasField<symbol!("auth_token"), Value =
Context::AuthToken>,
{
    fn auth_token(context: &Context) -> &Context::AuthToken {
        context.get_field(PhantomData)
    }
}
```

The `#[cgp_getter]` Macro

The `cgp` crate offers the `#[cgp_getter]` macro, which automatically derives implementations like `UseFields`. As an extension of `#[cgp_component]`, it provides the same interface and generates the same CGP component traits and blanket implementations.

With `#[cgp_getter]`, you can define accessor traits and seamlessly use `UseFields` directly in the component wiring, eliminating the need for manual implementations:

```
#[cgp_getter]
pub trait HasApiBaseUrl {
    fn api_base_url(&self) -> &String;
}

#[cgp_getter]
pub trait HasAuthToken: HasAuthTokenType {
    fn auth_token(&self) -> &Self::AuthToken;
}

#[cgp_context]
#[derive(HasField)]
pub struct ApiClient {
    pub api_base_url: String,
    pub auth_token: String,
}

delegate_components! {
    ApiClientComponents {
        ErrorTypeProviderComponent:
            UseAnyhowError,
        ErrorRaiserComponent:
            UseDelegate<RaiseApiErrors>,
        MessageTypeProviderComponent:
            UseType<String>,
        MessageIdTypeProviderComponent:
            UseType<u64>,
        AuthTokenTypeProviderComponent:
            UseType<String>,
        [
            ApiBaseUrlGetterComponent,
            AuthTokenGetterComponent,
        ]: UseFields,
        MessageQuerierComponent:
            ReadMessageFromApi,
    }
}
```

Compared to `#[cgp_auto_getter]`, `#[cgp_getter]` follows the same wiring process as other CGP components. To achieve the same outcome as `#[cgp_auto_getter]`, the only additional step required is delegating the getter component to UseFields within `delegate_components!`.

The primary advantage of using `#[cgp_getter]` is the ability to define custom accessor providers that can retrieve fields from the context in various ways, as we will explore in the next section.

Like `#[cgp_auto_getter]`, `#[cgp_getter]` can also be used with accessor traits containing multiple methods. This makes it easy to upgrade a trait, such as `HasApiClientFields`, to use `#[cgp_getter]` if custom accessor providers are needed in the future:

```rust
#[cgp_getter]
pub trait HasApiClientFields: HasAuthTokenType {
    fn api_base_url(&self) -> &String;

    fn auth_token(&self) -> &Self::AuthToken;
}
```

Static Accessors

One advantage of defining minimal accessor traits is that it allows the implementation of custom accessor providers that do not necessarily read field values from the context. For instance, we can create *static accessor* providers that always return a global constant value.

Static accessors are useful when we want to hard-code values for a specific context. For example, we might define a production `ApiClient` context that always uses a fixed API URL:

```rust
use std::sync::OnceLock;

#[cgp_new_provider]
impl<Context> ApiBaseUrlGetter<Context> for UseProductionApiUrl {
    fn api_base_url(_context: &Context) -> &String {
        static BASE_URL: OnceLock<String> = OnceLock::new();

        BASE_URL.get_or_init(|| "https://api.example.com".into())
    }
}
```

In this example, the `UseProductionApiUrl` provider implements `ApiBaseUrlGetter` for any context type. Inside the `api_base_url` method, we define a `static` variable `BASE_URL` using `OnceLock<String>`. This allows us to initialize the global variable exactly once, and it remains constant throughout the application.

`OnceLock` is especially useful since constructors like `String::from` are not `const` fn in Rust. By using `OnceLock::get_or_init`, we can run non-const constructors at runtime while still benefiting from compile-time guarantees. The static variable is scoped within the method, so it is only accessible and initialized by the provider.

With `UseProductionApiUrl`, we can now define a production `ApiClient` context, as shown below:

```
#[cgp_context]
#[derive(HasField)]
pub struct ApiClient {
    pub auth_token: String,
}

delegate_components! {
    ApiClientComponents {
        ErrorTypeProviderComponent:
            UseAnyhowError,
        ErrorRaiserComponent:
            UseDelegate<RaiseApiErrors>,
        MessageIdTypeProviderComponent:
            UseType<u64>,
        MessageTypeProviderComponent:
            UseType<String>,
        AuthTokenTypeProviderComponent:
            UseType<String>,
        ApiBaseUrlGetterComponent:
            UseProductionApiUrl,
        AuthTokenGetterComponent:
            GetAuthToken,
        MessageQuerierComponent:
            ReadMessageFromApi,
    }
}
```

In the component wiring, we specify `UseProductionApiUrl` as the provider for `ApiBaseUrlGetterComponent`. Notably, the `ApiClient` context no longer contains the `api_base_url` field.

Static accessors are particularly useful for implementing specialized contexts where certain fields must remain constant. With this approach, constant values don't need to be passed around as part of the context during runtime, and there's no concern about incorrect values being assigned at runtime. Additionally, because of the compile-time wiring, this method may offer performance benefits compared to passing dynamic values during execution.

Using `HasField` Directly Inside Providers

Since the `HasField` trait can be automatically derived by contexts, some developers may be tempted to forgo defining accessor traits and instead use `HasField` directly within the providers. For example, one could remove `HasApiBaseUrl` and `HasAuthToken` and implement `ReadMessageFromApi` as follows:

```
#[cgp_new_provider]
impl<Context> MessageQuerier<Context> for ReadMessageFromApi
where
    Context: HasMessageIdType<MessageId = u64>
        + HasMessageType<Message = String>
        + HasAuthTokenType
        + HasField<symbol!("api_base_url"), Value = String>
        + HasField<symbol!("auth_token"), Value = Context::AuthToken>
        + CanRaiseError<reqwest::Error>
        + CanRaiseError<ErrStatusCode>,
    Context::AuthToken: Display,
{
    fn query_message(context: &Context, message_id: &u64) -> Result<String,
Context::Error> {
        let client = Client::new();

        let url = format!(
            "{}/api/messages/{}",
            context.get_field(PhantomData::<symbol!("api_base_url")>),
            message_id
        );

        let response = client
            .get(url)
            .bearer_auth(context.get_field(PhantomData::<symbol!
("auth_token")>))
            .send()
            .map_err(Context::raise_error)?;

        let status_code = response.status();

        if !status_code.is_success() {
            return Err(Context::raise_error(ErrStatusCode { status_code }));
        }

        let message_response: ApiMessageResponse =
response.json().map_err(Context::raise_error)?;

        Ok(message_response.message)
    }
}
```

In the example above, the provider `ReadMessageFromApi` requires the context to implement `HasField<symbol!("api_base_url")>` and `HasField<symbol!("auth_token")>`. To preserve the original behavior, we add constraints ensuring that the `api_base_url` field is of type `String` and that the `auth_token` field matches the type of `Context::AuthToken`.

When using `get_field`, since there are multiple `HasField` instances in scope, we need to fully qualify the field access to specify which field we want to retrieve. For example, we call `context.get_field(PhantomData::<symbol!("api_base_url")>)` to access the `api_base_url` field.

However, while the direct use of `HasField` is possible, it does not necessarily simplify the code. In fact, it often requires more verbose specifications for each field. Additionally, using `HasField` directly necessitates explicitly defining the field types. In contrast, with custom accessor traits like `HasAuthToken`, we can specify that a method returns an abstract type like `Self::AuthToken, which prevents accidental access to fields with the same underlying concrete type.

Using `HasField` directly also makes the provider less flexible if the context requires custom access methods. For instance, if we wanted to put the `api_base_url` field inside a separate `ApiConfig` struct, we would run into difficulties with `HasField`:

```
pub struct Config {
    pub api_base_url: String,
    // other fields
}

pub struct ApiClient {
    pub config: Config,
    pub auth_token: String,
    // other fields
}
```

In this case, an accessor trait like `HasApiUrl` would allow the context to easily use a custom accessor provider. With direct use of `HasField`, however, indirect access would be more cumbersome to implement.

That said, using `HasField` directly can be convenient during the initial development stages, as it reduces the number of traits a developer needs to manage. Therefore, we encourage readers to use `HasField` where appropriate and gradually migrate to more specific accessor traits when necessary.

The `UseField` Pattern

In the previous chapter, we were able to implement context-generic accessor providers like `GetApiUrl` and `UseFields` without directly referencing the concrete context. However, the field names, such as `api_url` and `auth_token`, were hardcoded into the provider implementation. This means that a concrete context cannot choose different *field names* for these specific fields unless it manually re-implements the accessors.

There are various reasons why a context might want to use different names for the field values. For instance, two independent accessor providers might choose the same field name for different types, or a context might have multiple similar fields with slightly different names. In these cases, it would be beneficial to allow the context to customize the field names instead of having the providers pick fixed field names.

To address this, the `cgp` crate provides the `UseField` marker type (note the lack of `s`, making it different from `UseFields`), which we can leverage to implement flexible accessor providers:

```
pub struct UseField<Tag>(pub PhantomData<Tag>);
```

Similar to the `UseDelegate` pattern, the `UseField` type acts as a marker for accessor implementations that follow the UseField pattern. Using `UseField`, we can define the providers as follows:

```
#[cgp_provider]
impl<Context, Tag> ApiBaseUrlGetter<Context> for UseField<Tag>
where
    Context: HasField<Tag, Value = String>,
{
    fn api_base_url(context: &Context) -> &String {
        context.get_field(PhantomData)
    }
}

#[cgp_provider]
impl<Context, Tag> AuthTokenGetter<Context> for UseField<Tag>
where
    Context: HasAuthTokenType + HasField<Tag, Value = Context::AuthToken>,
{
    fn auth_token(context: &Context) -> &Context::AuthToken {
        context.get_field(PhantomData)
    }
}
```

Compared to `UseFields`, the implementation of `UseField` is parameterized by an additional `Tag` type, which represents the field name we want to access. The structure of the implementation is almost the same as before, but instead of using `symbol!` to

directly reference the field names, we rely on the `Tag` type to abstract the field names.

Deriving `UseField` from `#[cgp_getter]`

The implementation of `UseField` on accessor traits can be automatically derived when the trait is defined with `#[cgp_getter]`. However, the derivation will only occur if the accessor trait contains exactly one accessor method. This is because, in cases with multiple methods, there is no clear way to determine which accessor method should utilize the `Tag` type specified in `UseField`.

By combining `#[cgp_getter]` with `UseField`, we can streamline the implementation of `ApiClient` and directly wire the accessor components within `delegate_components!`:

```
#[cgp_getter]
pub trait HasApiBaseUrl {
    fn api_base_url(&self) -> &String;
}

#[cgp_getter]
pub trait HasAuthToken: HasAuthTokenType {
    fn auth_token(&self) -> &Self::AuthToken;
}

delegate_components! {
    ApiClientComponents {
        ErrorTypeProviderComponent: UseAnyhowError,
        ErrorRaiserComponent: UseDelegate<RaiseApiErrors>,
        MessageIdTypeProviderComponent: UseType<u64>,
        MessageTypeProviderComponent: UseType<String>,
        AuthTokenTypeProviderComponent: UseType<String>,
        ApiBaseUrlGetterComponent: UseField<symbol!("api_base_url")>,
        AuthTokenGetterComponent: UseField<symbol!("auth_token")>,
        MessageQuerierComponent: ReadMessageFromApi,
    }
}
```

In this wiring example, `UseField<symbol!("api_base_url")>` is used to implement the `ApiBaseUrlGetterComponent`, and `UseField<symbol!("auth_token")>` is used for the `AuthTokenGetterComponent`. By explicitly specifying the field names in the wiring, we can easily change the field names in the `ApiClient` context and update the wiring accordingly.

Conclusion

In this chapter, we explored various ways to define accessor traits and implement accessor providers. The `HasField` trait, being derivable, offers a way to create context-generic accessor providers without directly accessing the context's concrete fields. The `UseField` pattern standardizes how field accessors are implemented, enabling contexts to customize field names for the accessors.

As we will see in later chapters, context-generic accessor providers allow us to implement a wide range of functionality without tying code to specific concrete contexts. This approach makes it possible to maintain flexibility and reusability across different contexts.